

Violent action detection and its prevention



Project Team

Sl. No.	Reg. No.	Student Name
1	19ETCS002112	Sanjana K
2	19ETCS002138	V Varshitha
3	19ETCS002139	Pravallika V
4	19ETCS002403	Mushkan Kumar
5	19ETCS002157	Rishika Saha

B. Tech. in Computer Science and Engineering

FACULTY OF ENGINEERING AND TECHNOLOGY

M. S. RAMAIAH UNIVERSITY OF APPLIED SCIENCES

Bengaluru -560 054

FACULTY OF ENGINEERING AND TECHNOLOGY

Certificate

*This is to certify that the Project titled “**Violent action detection and its prevention**” is a bonafide work carried out in the **Department of Computer Science and Engineering** by Varshitha V, Sanjana K, Pravallika V, Mushkan Kumar and Rishika Saha bearing Reg. No. 19ETCS002138,19ETCS002112,19ETCS002139,19ETCS002403 ,19ETCS002157. respectively in partial fulfilment of requirements for the award of B. Tech. Degree in Computer Science and Engineering of Ramaiah University of Applied Sciences.*

April- 2023

<Dr. Jyothi A P>

**<Dr. Rinki Sharma>
Professor and Head – Dept. of CSE**

**<Dr. Dilip Kumar Mahanti>
Professor and Dean-FET**



Declaration

Violent action detection and its prevention

The project work is submitted in partial fulfilment of academic requirements for the award of **B. Tech.** Degree in the **Department of Computer Science and Engineering** of the Faculty of **Engineering and Technology** of Ramaiah University of Applied Sciences. The project report submitted herewith is a result of our own work and in conformance to the guidelines on plagiarism as laid out in the University Student Handbook. All sections of the text and results which have been obtained from other sources are fully referenced. We understand that cheating and plagiarism constitute a breach of University regulations, hence this project report has been passed through plagiarism check and the report has been submitted to the supervisor.

Sl. No.	Reg. No.	Student Name	Signature
1	19ETCS002112	Sanjana K	
2	19ETCS002138	Varshitha V	
3	19ETCS002139	Pravallika V	
4	19ETCS002403	Mushkan Kumar	
5	19ETCS002157	Rishika Saha	

Date : **11th April 2023**

Acknowledgements

It is with extreme pleasure and pride that we present our B-Tech. dissertation titled *“Violent action detection and its prevention.”*. We would like to express our sincere thanks and gratitude to the following people, who stood by us throughout, helping us with much required inputs, guidance, knowledge and supported us.

We take great pleasure to express our sincere thanks and gratitude to academic project guide **Dr. Jyothi A P**, for her support, guidance and suggestions throughout the project which is leading this project for the completion.

We express our sincere thanks to, **Dr. Dilip Kumar Mahanti**, our respected Dean and to **Dr. Rinki Sharma**, Professor and Head of Department of Computer Science and Engineering, for their kind cooperation and support toward out dissertation, and to the management of Ramaiah University of Applied Science for their continued support. We are thankful to the staff members of the Computer Science and Engineering, RUAS for giving us good support and suggestion.

Lastly, we would like to thanks our parents and friends for their continued support, encouragement and motivation and God for paving our way of success in this object.

Abstract

Face detection and action detection technologies, for example, are now being used to help prevent crime. Action detection technology is a useful investigative tool for detecting and preventing criminal activity, reducing an imminent threat to health or safety, protecting the public, and improving security and safety. While face detection, when combined with violent action detection, plays an important role in catching perpetrators involved in the crime. When combined with human analysis and additional investigation, this technology is a valuable tool for solving crimes and increasing public safety.

The scope of this project is that it will be used for crime detection and to improve today's security with the help of technology. It does not only limit to preventing crimes but also offer a better security service for the people.

Face and action detection is a promising example of a new development in crime prevention technology that has gained popularity in recent years. These technologies can be helpful in reducing violent crime by identifying people and their behaviour in public areas using artificial intelligence algorithms.

This system can serve as a deterrent to violent crime in addition to spotting possible threats. Potential criminals may be less reluctant to carry out violent crimes in public places if they are aware that they are being observed by sophisticated technology, which would lower the overall crime rate.

Advanced versions of this system must make sure that data is gathered and utilised in a way that is open, responsible, and has clear regulations about data preservation and access. In order to make sure that these systems do not prejudice any specific persons or groups, it is also vital that they undergo frequent audits and testing.

Overall, face and action detection technology hold out considerable promise for reducing violent crime, but it is crucial to weigh this potential against ethical issues related to discrimination and privacy. These systems have the potential to be an important instrument in preserving public safety and lowering crime rates with proper planning and execution.



Table of Contents

Acknowledgements	2
Abstract	3
List of Figures	8
List of Tables	11
1. Introduction	12
1.1 Introduction	12
1.2 Literature Survey	14
1.3 Conclusion	18
2. Background Theory	20
2.1 Background Theory:	20
2.1.1 Python	20
2.1.2 TensorFlow	20
2.1.3 Twilio (TwiML)	21
2.1.4 Wing IDE	21
2.1.5 OpenCV	21
2.1.6 HaarCascade	21
2.2 Background of Existing Application	22
2.2.1 Real-Time Violent Action Recognition using Key Frames Extraction and DL	22
2.2.2 Application-Driven AI Paradigm for Hand-held Action Detection	23
2.3 Conclusion	25
3. Aim and Objectives	26
3.1 Title	26
3.2 Aim	26
3.3 Objectives	26
3.4 Functional Requirements	27
3.5 Non- Functional requirements	28

3.6 Methods and Methodologies	29
3.7 Conclusion	31
4. Problem Solving	32
4.1 Design	32
4.1.1 Use Case Diagram	33
4.1.2 Flowchart of the general working of the system	33
4.2 Implementation	34
4.2.1 Face Detection	34
4.2.2 Violent Action Detection	35
4.2.2.1 Training the model	35
4.2.2.2 Detection of violence in the real time video	39
4.2.3 Notification of violence	40
4.2.3.1 Sounding a security alarm on violence detected	41
4.2.3.2 Location access using GPS - getLocation() function	41
4.2.3.3 Sending SMS and whatsapp messages	42
4.2.3.4 Making voice calls	43
4.3 Testing	45
4.4 Performance Analysis	46
5. Results	47
5.1 Screenshot	47
5.1.1 Training Model	47
5.1.1.1 Data Extraction	47
5.1.1.2 Basic Model	48
5.1.1.3 Basic Model Continuation	48
5.1.1.4 Parameters that can be trained	57
5.1.1.5 Training of model	58



M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)

5.1.1.6 Training and validation loss	60
5.1.1.7 Training and validation accuracy	61
5.1.1.8 Evaluation of network	62
5.1.2 Case 1: Violent actions detected	63
5.1.2.1 Video Capture	63
5.1.2.2 Face Detection	65
5.1.2.3 Action Detection with Labels	65
5.1.2.4 Notification	67
5.1.3 Case 2: Non Violent actions detected	70
5.1.3.1 Video Capture	70
5.1.3.2 Face Detection	71
5.1.3.3 Action Detection with Labels	71
5.1.3.4 Notification	72
5.2 Summary	72
6. Project Costing	74
6.1 Project Cost Estimation	74
6.2 Conclusion	74
7. Conclusions and Suggestions for Future Work	75
7.1 Conclusion	75
7.2 Suggestion for future work	76
References	78
Appendix	80

List of Figures

Figure 1 Architecture of sequential CNN for violence detection	22
Figure 2 Fine detection model's annotation	24
Figure 3 Coarse detection model's annotation respectively.	24
Figure 4 Images containing false positive target, mistakes made as a result of intrinsic shortages.	25
Figure 5 Use Case Diagram	33
Figure 6 Flowchart of the system	33
Figure 7 Loop for image classification	35
Figure 8 A sample of violent dataset	36
Figure 9 A sample of non-violent dataset	37
Figure 10 Loop for extraction of data from the classes	37
Figure 11 Creation of Model Checkpoint	38
Figure 12 model.fit() function	38
Figure 13 Accessing the webcam and recording the video	39
Figure 14 Cloning, resizing and prediction of action.	40
Figure 15 Code snippet that plays the MP3 alarm file	41
Figure 16 The getLocation() function	42
Figure 17 The sendMessage() function	43
Figure 18 The makeCall() function	44
Figure 19 TwiML Bin containing TWIML code for response	44
Figure 20 Data Extraction	47
Figure 21 Basic Model	48
Figure 22 Basic Model Continuation1	49
Figure 23 Basic Model Continuation2	50

Figure 24 Basic Model Continuation3	51
Figure 25 Basic Model Continuation4	52
Figure 26 Basic Model Continuation5	53
Figure 27 Basic Model Continuation6	54
Figure 28 Basic Model Continuation7	55
Figure 29 Basic Model Continuation8	56
Figure 30 Basic Model Continuation9	57
Figure 31 Parameters that can be trained	57
Figure 32 Training of model	58
Figure 33 loss, accuracy, val_loss and val_accuracy of epoch1 and starting of epoch2	58
Figure 34 loss, accuracy, val_loss and val_accuracy of epoch14 and starting of epoch15	59
Figure 35 loss, accuracy, val_loss and val_accuracy of epoch25	59
Figure 36 Predicting the best epochs, calculating the accuracy of training and testing data and loss of training and testing data	59
Figure 37 Graph for training and validation loss	60
Figure 38 Graph for Training and validation accuracy	61
Figure 39 Evaluation of network	62
Figure 40 Graphical plot for Confusion matrix of model evaluation	63
Figure 41 Screenshot while video capturing 1	64
Figure 42 Screenshot while video capturing 2	64
Figure 43 Faces saved when face recognition algorithm was applied	65
Figure 44 Action Detection with Labels	65
Figure 45 Violence detection with label 1	66
Figure 46 Violence detection with label 2	67
Figure 47 Violence identified and the location is fetched	67
Figure 48 Call and messages are sent with location when violence is detected and alarm is triggered.	68

Figure 49 Call received on the recipient's phone	68
Figure 50 Whatsapp message	69
Figure 51 Text Message	69
Figure 52 Screenshot while video capturing 1	70
Figure 53 Screenshot while video capturing 2	71
Figure 54 Faces saved when face recognition algorithm was applied	71
Figure 55 Non Violence detection with label	72
Figure 56 The actions are non violent. No notifications are sent.	72
Figure 57 Snippet1	80
Figure 58 Snippet2	80
Figure 59 Snippet3	81
Figure 60 Snippet4	82
Figure 61 Snippet5	83
Figure 62 Snippet6	84
Figure 63 Snippet7	85
Figure 64 Snippet8	86
Figure 65 Snippet9	87
Figure 66 Import all the required libraries	87
Figure 67 Webcam Accessing	88
Figure 68 Face Recognition	89
Figure 69 Action Recognition Snippet1	90
Figure 70 Action Recognition Snippet2	91
Figure 71 Location Accessing	92
Figure 72 Code for calls and Messages	92
Figure 73 Alarm Triggering	93

List of Tables

Table 1 Functional Requirements	27
Table 2 Non-Functional Requirements	28
Table 3 Method and Methodology	29,30,31
Table 4 Testing	45,46
Table 5 Cost estimation table	74

1. Introduction

1.1 Introduction

The advent of technology has revolutionized virtually every aspect of human life, including the way we approach crime prevention. With new and advanced tools constantly being developed, it is now possible to detect potential security threats in real-time and take pre-emptive measures to prevent violent crime. One such technology that has gained traction in recent years is face and action detection. This innovative approach utilizes artificial intelligence algorithms to analyse and identify individuals' facial expressions and body language in public spaces, with the aim of detecting signs of aggression or violent behaviour. By detecting these signs early on, law enforcement officials can take immediate action to prevent incidents from escalating and endangering public safety.

One of the key advantages of face and action detection technology is its ability to operate in real-time, which significantly reduces response times in the event of a threat. Furthermore, these systems can be integrated with other technologies such as facial recognition and video surveillance, to provide a comprehensive view of public spaces and potential risks. However, the use of this technology raises important ethical concerns around privacy and discrimination, which must be addressed to ensure that the benefits of the technology are not outweighed by the potential risks.

Despite the ethical concerns, face and action detection technology shows great promise in preventing violent crime and maintaining public safety. With careful design and implementation, these systems could become an essential tool in crime prevention, reducing crime rates and improving the quality of life for citizens. As technology

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)
continues to evolve, it is important to remain vigilant and ensure that new technologies are developed and implemented in an ethical and responsible manner.

Face and activity detection technology has become more crucial recently as a tool for enhancing public safety and deterring crime. In public places, this technology may be able to identify potential signs of aggression or violent behaviour in people's body language and facial expressions, alerting law enforcement officials to take pre-emptive action to stop the situation from getting worse. However, the employment of this technology raises important ethical issues of privacy, bias, and accuracy.

Privacy is a significant issue when using face and activity detection systems. People have a right to expect a certain level of privacy while they are in public locations, and using this technology may be viewed as intrusive. There's a chance that someone would be mistakenly categorised as a danger, which would force law enforcement to take unwarranted action. Additionally, there are concerns that this technology may be used against people's rights to privacy by following them around and keeping an eye on what they do without their consent.

Despite these concerns, face and activity detection technology has demonstrated a great deal of promise in terms of deterring crime. It may be used with other technology, such as facial recognition and video surveillance, to provide a full picture of public situations and potential dangers. Furthermore, it provides real-time capabilities that significantly speed up response times in the event of a threat. It may be used with other technology, such as facial recognition and video surveillance, to provide a full picture of public situations and potential dangers. Furthermore, it provides real-time capabilities that significantly speed up response times in the event of a threat. Face and action detection technologies must be developed and put into use with strong ethical ideals in mind in order to overcome these ethical issues. As part of this, it must be made sure that data is

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)
gathered and utilised in a way that is open, responsible, and has clear rules for data preservation and access. Additionally, it is essential that these systems undergo frequent audits and testing to make sure that they do not prejudice any certain racial or ethnic groups.

To sum up, face and action detection technology has the potential to be a useful tool in reducing violent crime, but its application must be carefully thought out to guarantee that it is in accordance with ethical and privacy norms. We can make the public area safer and more secure for everyone by weighing the advantages of this technology against these crucial factors.

1.2 Literature Survey

Technology for face and activity detection is developing quickly and is becoming more and more popular in the fields of public safety and criminal prevention. In order to recognise symptoms of hostility or violent conduct, this technology uses artificial intelligence algorithms to study and identify people's body language and facial expressions in public settings. With the intention of reducing violent crime, the technology has been used in a variety of locations, such as airports, retail centres, and public transportation networks. This review of the literature on face and action detection technology and its use in preventing violent crime seeks to provide readers an overview of the field.

Facial recognition and detection in this the technology for facial recognition has a wide range of uses, including security and crime prevention. The accuracy of the technology has, however, been a significant worry, notably in instances of false positives and negatives. In order to increase the accuracy of the technology, researchers have developed more advanced algorithms and techniques. To increase the algorithm's

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)
capacity for precise facial recognition, some of these strategies include merging data from several sources and deep learning techniques.

Face detection technology, which focuses on recognising faces in photos or videos, has also been created in addition to facial recognition technology. Real-time facial recognition algorithms may be used to recognise people in public places and can be combined with other technologies to identify possible threats. The algorithms recognise facial characteristics such as the eyes, nose, and mouth after determining if there are any faces in the input image. The generated information may be used to track people's activities and behaviours, identify specific persons or groups of people, and identify possible threats.

The goal of action detection technology is to spot motions and activities in movies and photos. With the use of this technology, it is possible to spot potentially hazardous circumstances, such as people brandishing weapons or acting violently. The complexity of human behaviour and movement makes it difficult to reliably recognise potentially harmful behaviours, which is one of the biggest hurdles in action detection. To solve this problem, researchers have created a number of algorithms, some of which use deep learning methods and combine data from several sources to get more accurate results.

Applications of Face and Action Detection Technology: Face and action detection technology has shown a lot of promise in the realm of preventing violent crime. The technology may be used with other security tools like facial recognition and video monitoring to give a complete picture of public areas and possible threats. For instance, to spot potentially harmful conduct and stop terrorist attacks, airports have employed face and activity detection technologies. Similar to how it has been utilised in public transportation and commercial centres to increase safety and deter violent crime.

Law enforcement has also utilised the technology to find suspects and solve crimes. Several people have been successfully prosecuted after using facial recognition technology to identify suspects in criminal investigations. However, there are substantial ethical questions about privacy, prejudice, and accuracy that have been brought up by the use of face recognition technology in police enforcement.

In Concerns of Ethics Several ethical questions are raised by the use of face and action detection technologies in the reduction of violent crime. Privacy is the main issue since people have a legitimate expectation of privacy in public areas. The application of this technology raises privacy concerns and has the potential to misidentify people as possible threats. Additionally, there is a chance that people's privacy rights might be violated if the technology is used to follow their whereabouts and observe their activities without their permission.

The potential that the algorithms utilised in this technology may be prejudiced towards particular groups and prompt unwarranted law enforcement action is another key worry. This bias could result from the algorithms' training data being biased toward particular demographic groups. The accuracy of the technology has also been questioned, with worries that false positives and negatives might result in discriminatory treatment and human rights abuses.

In addition to the ethical concerns, there are also technical challenges that need to be addressed in the development and implementation of face and action detection technology for violent crime prevention. One of the significant challenges is the integration of different technologies and data sources to achieve more comprehensive surveillance and threat detection capabilities. The technology requires a robust infrastructure to support its use, including high-speed connectivity, data storage, and processing capabilities.

The creation of algorithms that can properly detect possible threats while avoiding false positives and negatives is another problem. To attain a high degree of accuracy in threat detection, this calls for complex machine learning algorithms and the fusion of several data sources. Furthermore, concerns about public acceptability and confidence in the technology are raised by the employment of face and activity detection technologies in public settings. Public apprehensions about the technology's precision, likelihood of abuse, and violation of privacy might reduce its efficacy in reducing violent crime. Therefore, it is essential to involve the general public in debates about technology use and address their concerns.

Despite these difficulties, technology for detecting faces and actions has already demonstrated promise in reducing violent crime. Its application in public transportation, retail centres, and airports has helped to identify possible threats and stop violent situations. The application of the technology in law enforcement investigations has resulted in the successful prosecution of dangerous offenders.

In summary, the field of face and action detection technology is developing quickly and has the potential to completely change how violent crime is prevented. However, its usage must be carefully weighed against people's civil freedoms and right to privacy. For the technology to be used in an ethical and successful manner in the future, technical difficulties, ethical issues, and public acceptability must be solved. Face and action detection technology has the potential to play a significant role in guaranteeing public safety and reducing violent crime with further research, development, and public involvement.

As a result, while face and action detection technology has shown considerable potential in the realm of violent crime prevention, it also presents serious ethical questions. Although the technology has the potential to reduce violent crime and increase public

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)
safety, its usage must be carefully weighed against people's civil liberties and privacy rights.

More investigation is required to create algorithms that are more accurate and fair as well as to solve privacy and discrimination issues. It is essential to make sure that technology is applied responsibly and that its usage is governed by transparent rules and guidelines. Additionally, there is a requirement for public involvement and education on the application of face and action detection technologies in the reduction of violent crime. The general people must be aware of how technology functions, how it is applied, and how their rights are safeguarded. This interaction can promote confidence in the technology and guarantee future success in its application.

Overall, even if face and action detection technology has the potential to be a useful weapon in the fight against violent crime, its application has to be carefully controlled to prevent undue interference with people's rights and to make sure that it is applied ethically and successfully.

1.3 Conclusion

Facial recognition systems can be used to prevent crime in a variety of ways. FaceIt is one of these methods. FaceIt is a face recognition system that can search through a crowd for a face and match it with a person's mugshot history. It is currently regarded as the most accurate facial recognition software available. Face matching from live security images, face recognition in photographs, and face recognition from sketches are some of the other results. These are all methods for comparing a face (of, say, a potential criminal) to a face in live security images, photographs, or sketches to see if there is a match.

The system for recognising faces is far from perfect. There are still numerous dangers to be considered. Some of the risks associated with facial recognition include increased



M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)

criminal opportunities, the possibility of falsely accusing innocent people, and a loss of privacy. This paper concludes that facial recognition opens up numerous opportunities to help prevent crime. However, there are still a number of issues that can arise when these techniques are used in the real world.

2. Background Theory

In this chapter, all theory related to proposed project including technical aspect and resources are explained. This section starts with discussion of Programming Languages and Framework Used, IDE's on which the proposed project is built. Then discussion will turn towards online Artificial Intelligent Engines available and the one Used in this project. In the end the specific domain in which these engines are used will be explained. In the end, the entire theory will be summarised via a summery.

2.1 Background Theory:

This section gives complete knowledge and understanding of different technologies and framework that were required in this project for its completion. The background Theory is listed below:

2.1.1 Python

Python is an object-oriented programming language that is interpreted and interactive. It includes modules, exceptions, dynamic typing, very high level dynamic data types, and classes. Beyond object-oriented programming, it supports procedural and functional programming. Python combines incredible power with very clear syntax. It has interfaces to many system calls and libraries, as well as to various window systems, and it can be extended in C or C++. Finally, Python is portable: it runs on many Unix variants, including Linux and macOS, as well as Windows.

2.1.2 TensorFlow

TensorFlow is a free and open-source machine learning and artificial intelligence software library. It can be used for a variety of tasks, but it focuses on deep neural network training and inference. TensorFlow is compatible with a wide range of programming languages, including Python, JavaScript, C++, and Java. This adaptability lends itself to a wide range of applications in a variety of industries.

2.1.3 Twilio (TwiML)

TwiML, or the Twilio Markup Language, is an XML-based language that tells Twilio how to handle events like incoming and outgoing calls, SMS messages, and MMS messages. TwiML will be used when building a Twilio application to communicate your desired actions to Twilio.

2.1.4 Wing IDE

Wingware's Wing Python IDE family of integrated development environments (IDEs) was designed specifically for the Python programming language, with support for Python code editing, testing, debugging, inspecting/browsing, and error checking.

2.1.5 OpenCV

OpenCV is a large open-source library for computer vision, machine learning, and image processing, and it now plays a significant role in real-time operation, which is critical in today's systems. It can process images and videos to identify objects, faces, and even human handwriting. Python can process the OpenCV array structure for analysis when combined with other libraries such as NumPy. We use vector space and perform mathematical operations on these features to identify image patterns and their various features.

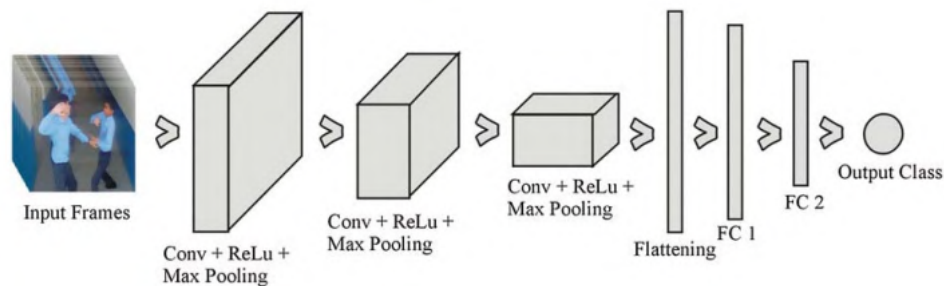
2.1.6 HaarCascade

The Haar cascade algorithm can detect objects in images regardless of their scale or location. This algorithm is not overly complex and can be executed in real time. We can train a haar-cascade detector to detect objects such as cars, bikes, buildings, fruits, and so on. Haar cascade employs the cascading window to compute features in each window and classify whether it is an object.

2.2 Background of existing applications

2.2.1 Real-Time Violent Action Recognition using Key Frames Extraction and DL:

The paper titled “Real-Time Violent Action Recognition Using Key Frames Extraction and Deep Learning” describes a fully automated system for detecting violent activity in surveillance videos. For feature selection and classification, the proposed system employs two convolutional neural network architectures, Sequential and Inception v4 CNN.



Architecture of sequential CNN for violence detection

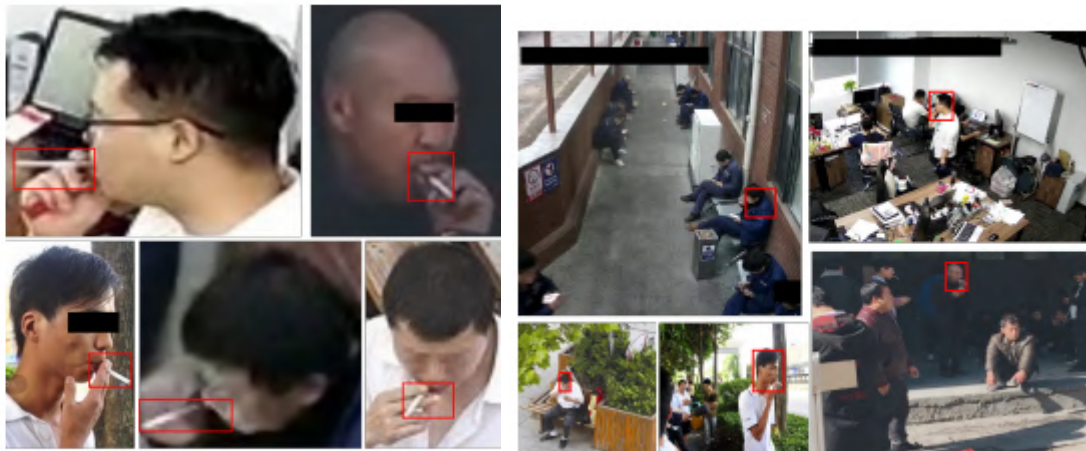
The study also prepares a violent activity dataset for training classification models and compares the proposed framework's performance on three benchmark datasets to state-of-the-art models. The paper emphasises the need for a fully automated system because the current semi-automated system for detecting violent activities through surveillance cameras is inefficient and requires continuous monitoring. The proposed system uses computer vision and machine learning to overcome this challenge.

The paper's related work section discusses previous research on video action detection,

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET) classifying fully automatic violence detection methods as trajectory-based, non-object-centric, and deep-learning-based techniques. Trajectory-based methods estimate object motion and extract video trajectory features. Methods that are not object-centric extract video features based on object behaviour rather than object motion. Deep neural networks are used in deep learning techniques to extract discriminative features from videos. The paper compares various trajectory-based methods based on feature extraction and classification techniques. On three benchmark datasets, the proposed system outperforms existing methods using deep-learning techniques. The proposed system could be useful for monitoring public places such as banks and markets, where violence can occur at any time.

2.2.2 Application-Driven AI Paradigm for Hand-held Action Detection:

This model proposes a coarse-to-fine method that consists of two stages. It uses two detection modules: the first module focuses mainly coarse object of the human doing an action, whereas the other focuses on the fine object in the image, which could be anything that the human is interacting with, for example, a cigarette, a knife, a football, and so on. This particular model focuses mainly on perfecting the detection of a human smoking a cigarette. Here, the human is the coarse object and the fine object will be the cigarette.



a)

b)

The above mentioned a) *fine detection model's annotation* and b) *coarse detection model's annotation* respectively.

This system utilised the basic object detection model YOLOv5 and faster rcnn to construct detection model and the fine detection model for their ablation study experiments.

For the coarse detection model's dataset, the developers collected images from public webs, manual simulation and real-word scenarios. The images in this dataset contain various gestures, angles, facial features, people of different gender and ages, smoking poses and so on. For the fine detection model's dataset, it is constructed by applying the coarse object detection and fine annotation. An additional dataset was also added to help recognize objects that are very likely to be classified as smoking cigarettes, even an unlit cigarette will be taken into account as a smoking cigarette. This enhanced the precision and recall of the model simultaneously.



Examples of images containing false positive target; mistakes made as a result of intrinsic shortages.

2.3 Conclusion

In this Chapter, All background knowledge for completion of this project are explained elaborately. Also, in each mentioned resource and technology, the reason for its use is also expressed. Languages like Python and Ide like Wing.ide and its use in this project is explained.

3. Aim and Objectives

This chapter focuses on the defined title and Aim of the project correctly and clearly. Later this chapter also includes the required objectives that needed to be fulfilled in order to complete this project. Functional Requirements are well documented in this section since it is required to design different diagrams leading to a complete view of the project. This is followed by methods and methodologies that tabulate the procedure at will be followed in order to complete the objectives. This section then ends with a summary.

3.1 Title

Violent action detection and prevention of violent crimes

3.2 Aim

To detect violent crime happening in real-time and prevent it from escalating using computer vision and data analytics.

3.3 Objectives

The objectives of the proposed Project are listed below:

1. The objective of this research is to detect human faces and violent actions occurring in a real-time video.
2. When an action in the video is detected as 'violent', an alarm is sounded in the location where the crime is taking place, in an attempt to stop the crime from escalating.
3. Notifying the security personnel on the university campus by sending an automated voice message and the location of the crime.
4. Notifying the nearest police station of the crime by sending an automated voice message and the location of the crime.

5. Deep Learning and computer vision will be used to achieve these objectives.

3.4 Functional Requirements

This project's functional requirements are mentioned below:

Sl. no	Functional Requirement	Description
1	FR 1	The system should detect the face of the person appearing in the real-time video.
2	FR 2	The system should detect the action happening in the real-time video.
3	FR 3	The system should classify the detected action as violent or not violent.
4	FR 4	The system should sound an alert alarm if the action detected was violent.
5	FR 5	The system should send an automated alert SMS and voice message to the authorities about the violent action when the action detected is violent.
6	FR 6	The system should send the message along with the exact location where the violent action is detected.

3.5 Non- Functional Requirements:

Sl. no	Non-Functional Requirement	Description
1	NFR 1	Performance: the system's ability to meet latency, throughput, and resource utilisation requirements.
2	NFR 2	Scalability: the system should handle a large amount of data.
3	NFR 3	Availability: the system should be available 24/7.
4	NFR 4	Security: The system should have the ability to prevent restricted actions
5	NFR 5	Reliability & user friendly: The extent to which the system will execute without failure for a specific period.

3.6 Method and Methodology

Table 2 Methods and Methodology

Objective No.	Statement of the Objective	Method/ Methodology	Resources Utilised
1	The objective of this research is to detect human faces and violent actions occurring in a real-time video.	1.1 Haar Cascade: Haar Cascade is an object detection algorithm to detect faces in images or videos. A cascade function is trained using many positive and negative images which can be later used to identify any object or face in other media. These trained files are available in the OpenCV GitHub repository. 1.2 Violent action detection: Using TensorFlow and openCV to train an action detection model and apply it to test data.	Took the reference from the internet along with a help of our mentor.
2	On detection of a crime, an alarm is sounded in the location where the crime is taking place, in an attempt	playsound library: Used to play an alarm sound when a violent action is detected.	Took the reference from the internet along with a help of our mentor.

	to stop the crime from escalating.		
3	Notifying the security personnel on the university campus by sending an automated voice message and the location of the crime.	3.1 Twilio software and the ‘Twilio’ module in Python: Used for sending automated voice messages and SMS to the higher authorities. 3.2 The Global Positioning System (GPS) and the ‘winrt’ and ‘Google’ modules in Python: The location of the crime will be accessed using GPS.	Took the reference from the internet along with a help of our mentor.
4	Notifying the nearest police station of the crime by sending an automated voice message and the location of the crime.	4.1 Twilio software and the ‘Twilio’ module in Python: Used for sending automated voice messages and SMS to the higher authorities. 4.2 The Global Positioning System (GPS) and the ‘winrt’ and ‘Google’ modules in Python: The location of the crime will be accessed using GPS	Took the reference from the internet along with a help of our mentor.
5	Deep Learning and computer vision will be used to achieve these objectives.	Real Life Violence Situations Dataset – –Contains violent videos collected from YouTube videos, street fights, etc.	Referred to Kaggle for the related dataset.

		–Contains non-violent videos collected from different human actions like eating, talking, walking, etc.	
--	--	---	--

3.7 Conclusion

This particular chapter focused on defining the title and Aim of the project correctly. Later this chapter included the required objectives that were required to be fulfilled to complete this project. Functional Requirements is documented successfully in this section followed by method and methodologies which is successfully tabulated in order to know the steps used in completing the objectives including resources used.

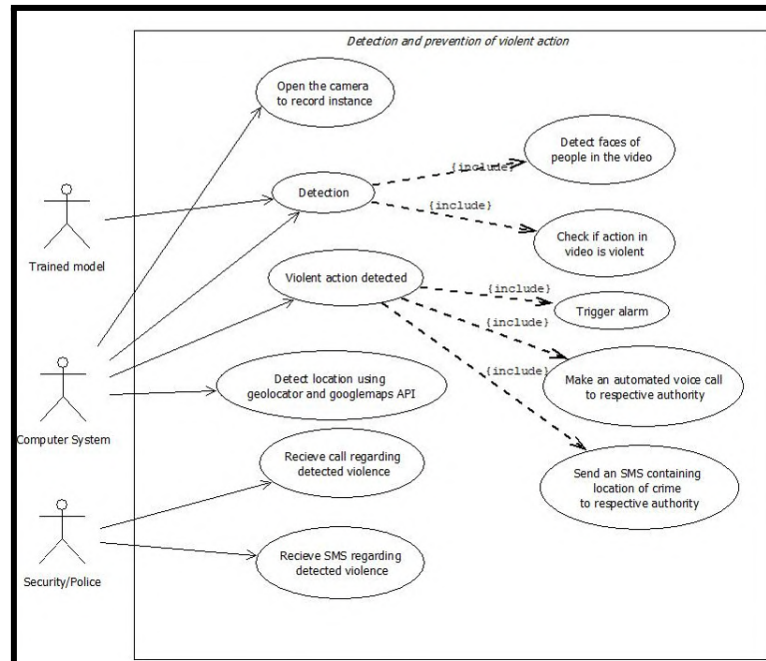
4. Problem Solving

In this section, the actual dissection of the project is done and each module is built piece by piece in order to complete the project. In the design section, the application has been built in accordance with functional requirements based on which diagrams like Use Case and Low-Level Sequence Diagrams are drawn. In the implementation section, snips of important code are displayed with their explanation given below. In the testing section, all functional requirements are tested and the result is analysed resulting in a status of the test condition. In the ending section, performance analysis is done on various factors such as the battery, booting time and time requirement for a major operation.

4.1 Design

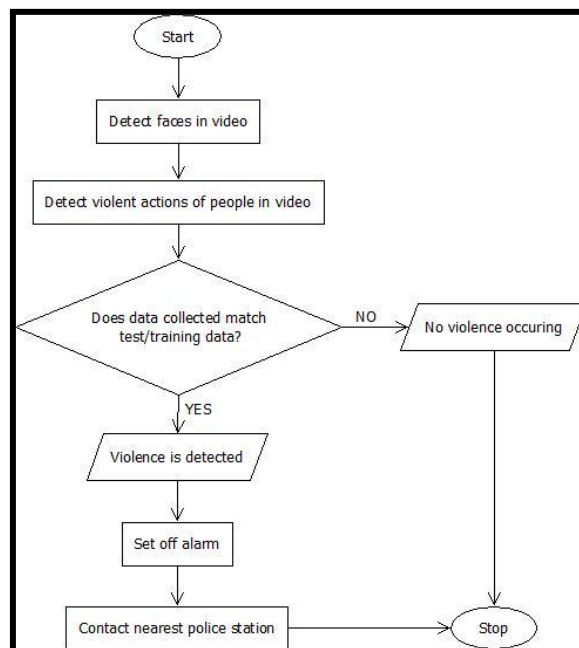
Design is necessary when it comes to development since it acts as a blueprint for the entire process from requirement making to finish (final product). Hence, in this section designs specific to this project like use case, sequence diagram etc. are attached.

4.1.1 Use Case Diagram



Use Case Diagram

4.1.2 Flowchart of the general working of the system



Flowchart of the system

4.2 Implementation

4.2.1 Face Detection:

A haar-cascade detector is trained to detect the faces of the people that appear in the video. The algorithm is given a lot of positive images consisting of faces, and a lot of negative images not consisting of any face to train on them. The model created from this training is available at the OpenCV GitHub repository.

In our project “**Detection of Violent Crimes** ” we have used the HAAR Cascade algorithm to detect the faces of the people appearing in the video.

In our project for the face detection code, we have first we have taken the path for the xml file in haar cascade. Next, we have defined the path where the image will be saved and also define the size of the images the device will capture. Next, we have also integrated the webcam camera in order to capture the video as well. Next, we have set a loop to take images using the open CV and then save it in the folder path as mentioned above in the code. The final results of the code include the face detection images as captured by the camera and the video of the same. The images taken in this part is again used to implement the action detection part of the project .

```
while count < 100:
    (_, im) = vs.read()
    gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 4)
    for (x, y, w, h) in faces:
        cv2.rectangle(im, (x, y), (x + w, y + h), (255, 0, 0), 2)
        face = gray[y:y + h, x:x + w]
        face_resize = cv2.resize(face, (width, height))
        print(cv2.imwrite('% s/% s.png' % (path1, count), face_resize))
    count += 1

cv2.imshow('OpenCV', im)
```

Loop for image classification

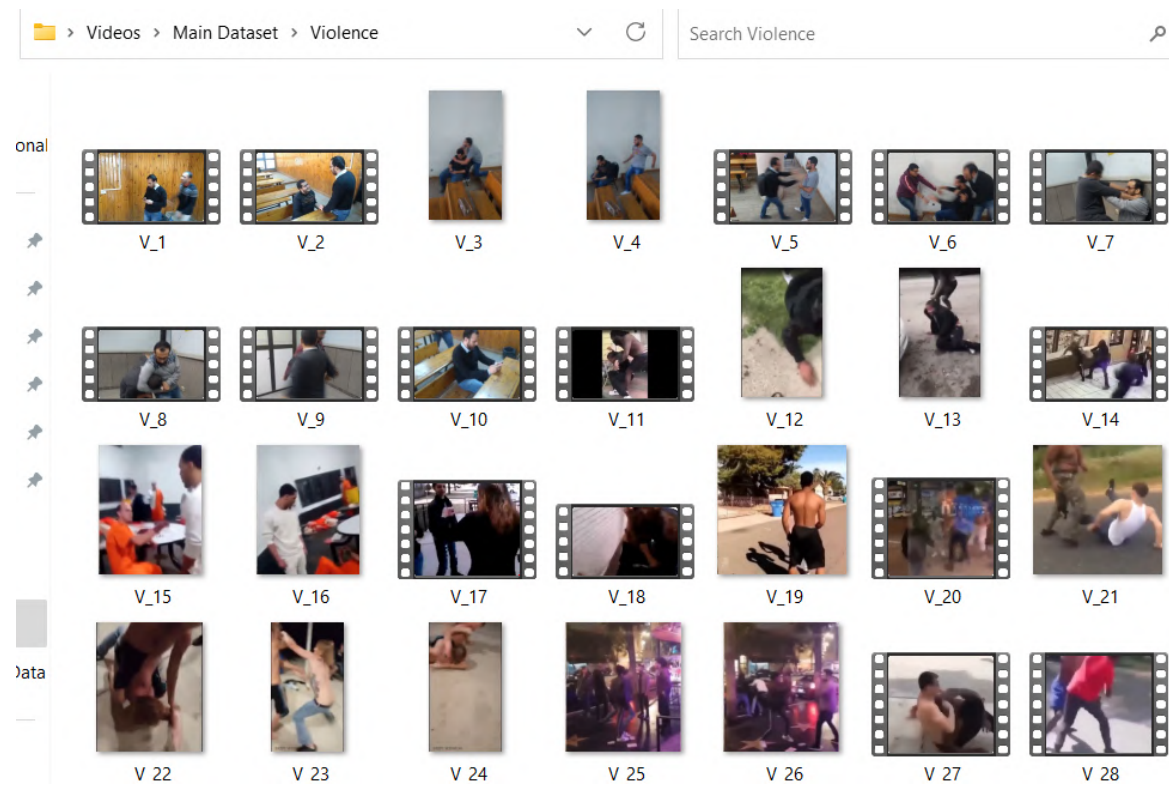
4.2.2 Violent action detection

MobileNetV2 model is trained using a dataset that contains real-time videos both violent and non-violent to detect whether a violent action has occurred in a given real-time video.

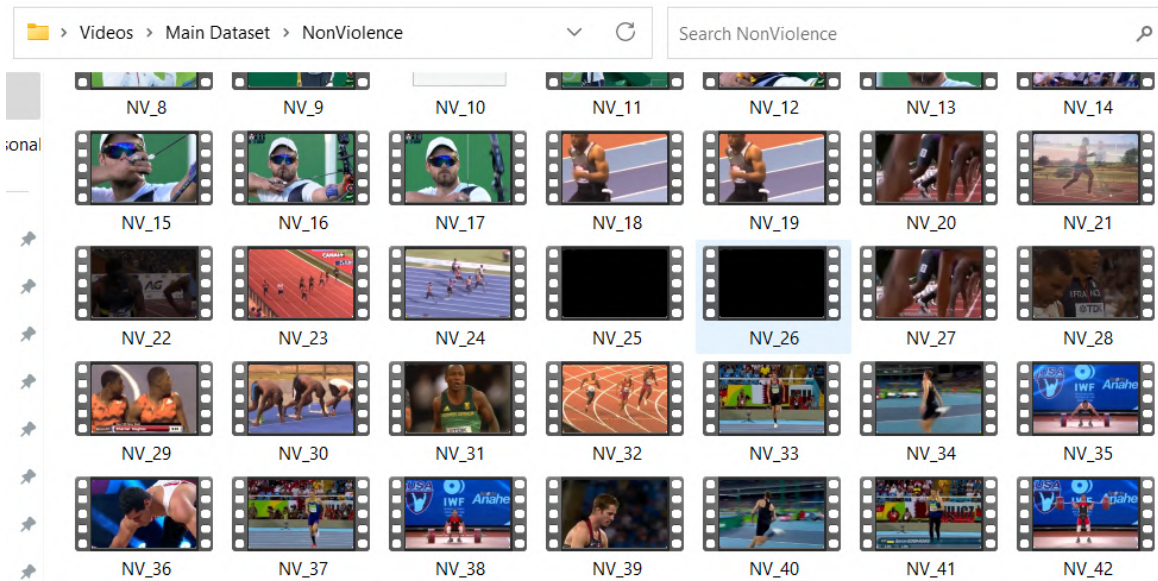
4.2.2.1 Training the model:

Seaborn and matplotlib modules are imported for data visualisation. cv2 module is imported to OpenCV packages. The tensorflow module is used for performing numerical computation. Scikit-learn (sklearn) module is used for classification purposes like identifying and categorising data based on patterns. imageio module is used for easy interface and writing a wide range of videos. math and numpy modules are imported to perform the mathematical tasks. Keras module is used for implementing neural networks and to support multiple backend neural network computations.

The video is converted into frames using cv2 and a loop is used to skip the frames to prevent creating duplication of frames and some image augmentations are applied. A variable is created to store the path of the dataset that is being used for training the model. The classes in the path are defined and are looped through to extract all the videos in their respective classes.



A sample of violent dataset



A sample of non violent dataset

```
for category in os.listdir(VideoDataDir):
    path = os.path.join(VideoDataDir, category)
    class_num = CLASSES.index(category)
    for i, video in enumerate(tqdm(os.listdir(path)[0:350])):
        frames = video_to_frames(path + '/' + video)
        for j, frame in enumerate(frames):
            X_original.append(frame)
            y_original.append(class_num)
```

Loop for extraction of data from the classes

To train and test the neural network the extracted dataset will be divided as training and testing datasets. We are using 25 epochs to train the developed model. All the layers that are required to train the model are loaded using a function. A checkpoint model “ModelWeights.h5” is created to hold all the exact values of the parameters created along with the date of creation.

```
checkpoint_filepath = 'ModelWeights.h5'

model_checkpoints = ModelCheckpoint(filepath=checkpoint_filepath,
                                    save_weights_only=True,
                                    monitor='val_loss',
                                    mode='min',
                                    verbose = 1,
                                    save_best_only=True)
```

Creation of Model Checkpoint

The model is trained by passing the indexes for training and testing indexes and respective datasets along with the number of epochs, callbacks and batch size as parameters to model.fit() function.

```
history = model.fit(X_train_nn ,y_train, epochs=epochs,
                    callbacks=callbacks,
                    validation_data = (X_test_nn, y_test),
                    batch_size=batch_size)
```

model.fit() function

After each epoch the values for loss, accuracy, validation loss and validation accuracy are calculated along with the improvement in the validation loss. All these values are updated to the checkpoint model after each epoch.

After training the model, two graphs are plotted, one for training and validation loss and the other for training and validation accuracy.

After training the model, the best epochs for the model are obtained and the accuracy and loss of both train and test datasets are obtained. The evaluation of the network is

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET) performed for the test dataset and the number of correct and wrong predictions are calculated and a corresponding confusion matrix is created.

After executing all the steps, the final trained model "modelnew.h5" is saved in the specified path so that it can be accessed later for detecting violence in the video.

4.2.2.2 Detection of violence in the real-time video using the trained model:

All the required modules required for this step are imported. The webcam is accessed using `cv2.VideoCapture(0)` and a video file with the name 'webcam.avi' with a codec of 'MP42' created using `VideoWriter_fourcc` is created using `VideoWriter()` function in the specified path.

```
# open the webcam video stream
webcam = cv2.VideoCapture(0)

# open output video file stream
video = VideoWriter('webcam.avi', VideoWriter_fourcc(*'MP42'), 25.0, (640, 480))
i=200
# main loop
while (i>0):
    # get the frame from the webcam
    stream_ok, frame = webcam.read()

    # if webcam stream is ok
    if stream_ok:
        # display current frame
        cv2.imshow('Webcam', frame)

        # write frame to the video file
        video.write(frame)
        i=i-1
```

Accessing the webcam and recording the video.

The model created in the training step, "modelnew.h5" is loaded and the video frames are read using read() function, if there is no frame then it is assumed as the end of the video. The output frame is cloned and it is converted from BGR to RGB. The action performed is violent or not is predicted for every frame of the video and is displayed at the top of the video. If the action performed is violent, "Violence: True" is displayed in red else "Violence: False" is displayed in green. The output image with the prediction is displayed using cv2.imshow().

```
# clone the output frame, then convert it from BGR to RGB
# ordering, resize the frame to a fixed 128x128, and then
output = frame.copy()

frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
frame = cv2.resize(frame, (128, 128)).astype("float32")
frame = frame.reshape(128, 128, 3) / 255

# make predictions on the frame and then update the predictions
# queue
preds = model.predict(np.expand_dims(frame, axis=0))[0]
print("preds",preds)
Q.append(preds)

# perform prediction averaging over the current history of
# previous predictions
results = np.array(Q).mean(axis=0)
i = (preds > 0.50)[0]
label = i
```

Cloning, resizing and prediction of action.

After the prediction is performed for the entire video the webcam is released using webcam.release(), the video is released using video.release() and all the opened windows are destroyed using cv2.destroyAllWindows().

4.2.3 Notification of violence

The authorities are notified if and only if the counter variable for detected violence is 1 or more than 1 in the violent action detection part of the code. Therefore, the notification part of the code is executed only if violent action has been detected.

4.2.3.1 Sounding a security alarm on detection of violence

The action detection code contains a counter that remains 0 means no violence was detected by the model in the video. If the counter is 1 or more than 1, then the action detection model detected violence occurring in the video and a security alarm is sounded in order to stop the crime from escalating. The Python package 'playsound' is used to play a locally stored MP3 alarm file.

```
from playsound import playsound
print('Playing sound...')
for i in range(3):
    playsound(r'C:\Users\Sanjana K\Downloads\mixkit-street-public-alarm-997.mp3')
print('Sound played...')
```

Code snippet that plays the MP3 alarm file

The audio file is played inside a for loop that runs 3 times.

4.2.3.2 Location access using GPS - the getLocation() function

For accessing the location where the camera, that is monitoring the crime taking place, one of Windows' runtime API, Windows.Devices.Geolocation is used in conjunction with the GoogleMaps API. The Geolocator class of the winrt API is used to obtain the current geographic location of the device. The 'locator' variable is a '_winrt_Windows_Devices_Geolocation.Geolocator' object and is used to create a Geoposition object. The 'pos' variable is a 'winrt_Windows_Devices_Geolocation.Geoposition' object and is used to access the

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)
latitude and longitude of the coordinate. This comprises of the getCoords() function. The getLoc() function is run concurrently using the asyncio module.

```
#location accessing
def getLocation():
    gmaps = googlemaps.Client(key='AIzaSyCxnnYtwIqzWlAuPsdMw11Qktg07V5l5qI')

    l = []
    async def getCoords():
        locator = wdg.Geolocator()
        pos = await locator.get_geoposition_async()
        print(locator)
        print('pos = ',pos)
        l.append(pos.coordinate.latitude)
        l.append(pos.coordinate.longitude)
        return [pos.coordinate.latitude, pos.coordinate.longitude]

    def getLoc():
        return asyncio.run(getCoords())
    print(getLoc())
    getLocation.url = "https://google.com/search?q="+str(l[0])+','+str(l[1])
    reverse_geocode_result = gmaps.reverse_geocode(tuple(l))
    getLocation.final_address = reverse_geocode_result[0]['formatted_address']
```

The getLocation() function

The reverse_geocode() of the googlemaps API is used to obtain a huge and complex list of the address of the latitude and longitude and other details. A tuple 'l', which contains latitude and the longitude of the location, is passed as argument to the reverse_geocode() function and the resulting list is stored in the variable 'reverse_geocode_result'. The 'formatted_address' index of the 'reverse_geocode_result' list contains the shorthand form of the address where the device is located. The url for the google map of that address is obtained by appending the common google-map link characters with the latitude and longitude of the location to be browsed.

4.2.3.3 Sending SMS and whatsapp messages - sendMessage() function

The function sendMessage() deals with sending messages. The software being used here is Twilio to send SMS and Whatsapp messages. The 'text1' variable consists of the text

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)
that is supposed to be the body of the message. The 'messages.create()' function of the client object contains of three arguments, namely, 'body' contains the message to be sent, 'from_' is the Twilio number that was obtained during creation of the Twilio account and is authorized to send messages and finally, 'to' argument which is the recipient's phone number to which the message is to be sent.

```
#sendMessage
def sendMessage():

    text1 = "Hello,\nHere is the location where the incident was detected along with the map URL of the location.\nAddress: "+getLocation.final_address+"\n\nLink:"+getLocation.url
    message = client.messages.create(
        body = text1,
        from_='+15073997123',
        to='+917204609477'
    )
    print(message.sid)

    message1 = client.messages.create(
        from_='whatsapp:+14155238886',
        body=text1,
        to='whatsapp:+917204609477'
    )

    print(message1.sid)
```

The sendMessage() function

The process remains almost entirely the same for sending whatsapp messages except that the 'from_' argument should be a separate Twilio number that has been approved for Whatsapp and the 'to' argument must contain the Whatsapp number of the recipient.

4.2.3.4 Making voice calls - the makeCall() function

```
#Make calls to number
def makeCall():
    call = client.calls.create(
        url="https://handler.twilio.com/twiml/EHa30e7d15c246d02d472ab2784e6ea03a",
        to="+917204609477",
        from_="+15073997123"
    )
    print(call.sid)
print("\nMessage and call SIDs:")
```

The makeCall() function

The function makeCall() deals with making phone calls from the registered Twilio number to any valid recipient number.

Twiml Bin

Properties

SID	EHa30e7d15c246d02d472ab2784e6ea03a	DATE CREATED	2023-03-29 22:06:15
URL	https://handler.twilio.com/twiml/EHa30e7d15c246d02d472ab2784e6ea03a	DATE UPDATED	2023-03-29 22:15:22

Configuration

FRIENDLY NAME

hihi

TWIML

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <Response>
3   <Pause length="5"/>
4   <Say>"Hello, a violent incident has been detected. Please refer to the message
   sent by SMS and Whatsapp for the location of the incident."</Say>
5   <Pause length="5"/>
6 </Response>
```

Twiml Bin containing TWIML code for response

The 'calls.create()' function of the client object consists of the following arguments, 'url' which is the URL to the Twilio's Twiml bin which contains the voice Twiml response when the recipient of the call answers the call, 'to' argument is the number of the recipient to whom the violent incident detected is to be reported and the 'from_' argument is the approved Twilio number capable of making outbound calls.

4.3 Testing

All functional requirement are tested and are tabulate below:

Table 3 Test Cases

SI No.	FR No.	Expected	Obtained	Result
1	FR 1	The system should detect the face of the person appearing in the real-time video.	The system successfully detect the face in a real time vedio	PASS
2	FR 2	The system should detect the action happening in the real-time video.	The system detected the action in the real time video.	PASS
3	FR 3	The system should classify the detected action as violent or not violent.	The system successfully classified the action as voilent or non violent	PASS
4	FR 4	In the case a violent action was detected, system should be able to generate an alert alarm.	On the detection of violent action system gives an alert alarm	PASS
5	FR 5	The system should send an automated alert SMS and voice message to the	The system successfully sent a alert message and	PASS

		authorities about the violent action when the action detected is violent	automated voice message to the higher authorities	
6	FR 6	The system should send the message along with the exact location where the violent action is detected.	The system on violent detection with automated message sends the exact location where the violent action takes place.	PASS

4.4 Performance Analysis

- The amount of time to train the action detection model by using 25 epochs was more than 40 minutes. This huge time consumption was noticed because the program was executed using only CPU and without GPU.
- In the current trained model though the accuracy is 96% it can be increased by increasing the size of the dataset along with increasing the number of epochs that are used to train the model.
- The dataset used to train the model is only a part of the dataset and not the whole dataset. Hence few wrong predictions are possible while predicting the actions.
- The notification methods that are adopted for reporting violence may not work properly if there are network issues

5. Results

5.1 Screenshots

5.1.1 Training Model:

5.1.1.2 Data Extraction

0%		0/350	[00:00<?, ?it/s]
0%		1/350	[00:00<05:13, 1.11it/s]
1%		2/350	[00:01<04:40, 1.24it/s]
1%		3/350	[00:01<02:54, 1.99it/s]
1% 1		4/350	[00:01<02:00, 2.88it/s]
1% 1		5/350	[00:02<01:32, 3.74it/s]
2% 1		6/350	[00:02<01:17, 4.45it/s]
2% 2		7/350	[00:02<01:05, 5.20it/s]
2% 2		8/350	[00:02<01:00, 5.68it/s]
3% 2		9/350	[00:02<00:54, 6.23it/s]
3% 2		10/350	[00:02<00:50, 6.72it/s]
3% 3		11/350	[00:02<00:46, 7.30it/s]
4% 3		13/350	[00:03<01:34, 3.57it/s]
4% 4		15/350	[00:03<01:07, 4.93it/s]
5% 4		17/350	[00:04<00:53, 6.22it/s]
5% 5		18/350	[00:04<01:20, 4.14it/s]
5% 5		19/350	[00:05<01:44, 3.17it/s]
6% 5		20/350	[00:05<02:04, 2.65it/s]
6% 6		21/350	[00:06<02:17, 2.40it/s]
6% 6		22/350	[00:06<02:31, 2.16it/s]
7% 6		23/350	[00:07<02:40, 2.03it/s]
7% 6		24/350	[00:07<02:06, 2.58it/s]
7% 7		25/350	[00:08<02:21, 2.30it/s]
7% 7		26/350	[00:08<02:33, 2.11it/s]
8% 8		28/350	[00:08<01:42, 3.14it/s]
8% 8		29/350	[00:09<01:29, 3.58it/s]
9% 8		30/350	[00:09<01:19, 4.04it/s]
9% 8		31/350	[00:09<01:12, 4.39it/s]
9% 9		32/350	[00:09<01:06, 4.81it/s]
9% 9		33/350	[00:09<01:02, 5.04it/s]
10% 9		34/350	[00:09<00:58, 5.37it/s]
10% #		35/350	[00:10<00:54, 5.79it/s]
10% #		36/350	[00:10<00:54, 5.79it/s]
11% #		37/350	[00:10<00:54, 5.79it/s]
11% #		38/350	[00:10<00:53, 5.78it/s]
11% #1		39/350	[00:10<00:53, 5.83it/s]
11% #1		40/350	[00:10<00:51, 5.99it/s]
12% #1		41/350	[00:11<00:50, 6.10it/s]

5.1.3 Basic Model:

Compiling model...
Model: "model"

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 128, 128, 3)]	0	[]
Conv1 (Conv2D)	(None, 64, 64, 32)	864	['input_1[0][0]']
bn_Conv1 (BatchNormalization)	(None, 64, 64, 32)	128	['Conv1[0][0]']
Conv1_relu (ReLU)	(None, 64, 64, 32)	0	['bn_Conv1[0][0]']
expanded_conv_depthwise (DepthwiseConv2D)	(None, 64, 64, 32)	288	['Conv1_relu[0][0]']
expanded_conv_depthwise_BN (BatchNormalization)	(None, 64, 64, 32)	128	['expanded_conv_depthwise[0][0]']
expanded_conv_depthwise_relu (ReLU)	(None, 64, 64, 32)	0	['expanded_conv_depthwise_BN[0][0]']
expanded_conv_project (Conv2D)	(None, 64, 64, 16)	512	['expanded_conv_depthwise_relu[0][0]']
expanded_conv_project_BN (BatchNormalization)	(None, 64, 64, 16)	64	['expanded_conv_project[0][0]']
block_1_expand (Conv2D)	(None, 64, 64, 96)	1536	['expanded_conv_project_BN[0][0]']
block_1_expand_BN (BatchNormalization)	(None, 64, 64, 96)	384	['block_1_expand[0][0]']
block_1_expand_relu (ReLU)	(None, 64, 64, 96)	0	['block_1_expand_BN[0][0]']
block_1_pad (ZeroPadding2D)	(None, 65, 65, 96)	0	['block_1_expand_relu[0][0]']
block_1_depthwise (DepthwiseConv2D)	(None, 32, 32, 96)	864	['block_1_pad[0][0]']

5.1.1.3 Basic Model Continuation1

block_1_depthwise_BN (BatchNormalization)	(None, 32, 32, 96)	384	['block_1_depthwise[0][0]']
block_1_depthwise_relu (ReLU)	(None, 32, 32, 96)	0	['block_1_depthwise_BN[0][0]']
block_1_project (Conv2D)	(None, 32, 32, 24)	2304	['block_1_depthwise_relu[0][0]']
block_1_project_BN (BatchNormalization)	(None, 32, 32, 24)	96	['block_1_project[0][0]']
block_2_expand (Conv2D)	(None, 32, 32, 144)	3456	['block_1_project_BN[0][0]']
block_2_expand_BN (BatchNormalization)	(None, 32, 32, 144)	576	['block_2_expand[0][0]']
block_2_expand_relu (ReLU)	(None, 32, 32, 144)	0	['block_2_expand_BN[0][0]']
block_2_depthwise (DepthwiseConv2D)	(None, 32, 32, 144)	1296	['block_2_expand_relu[0][0]']
block_2_depthwise_BN (BatchNormalization)	(None, 32, 32, 144)	576	['block_2_depthwise[0][0]']
block_2_depthwise_relu (ReLU)	(None, 32, 32, 144)	0	['block_2_depthwise_BN[0][0]']
block_2_project (Conv2D)	(None, 32, 32, 24)	3456	['block_2_depthwise_relu[0][0]']
block_2_project_BN (BatchNormalization)	(None, 32, 32, 24)	96	['block_2_project[0][0]']
block_2_add (Add)	(None, 32, 32, 24)	0	['block_1_project_BN[0][0]', 'block_2_project_BN[0][0]']
block_3_expand (Conv2D)	(None, 32, 32, 144)	3456	['block_2_add[0][0]']
block_3_expand_BN (BatchNormalization)	(None, 32, 32, 144)	576	['block_3_expand[0][0]']
block_3_expand_relu (ReLU)	(None, 32, 32, 144)	0	['block_3_expand_BN[0][0]']
block_3_pad (ZeroPadding2D)	(None, 33, 33, 144)	0	['block_3_expand_relu[0][0]']

Basic Model Continuation2

block_3_depthwise (DepthwiseConv2D)	(None, 16, 16, 144)	1296	['block_3_pad[0][0]']
block_3_depthwise_BN (BatchNormalization)	(None, 16, 16, 144)	576	['block_3_depthwise[0][0]']
block_3_depthwise_relu (ReLU)	(None, 16, 16, 144)	0	['block_3_depthwise_BN[0][0]']
block_3_project (Conv2D)	(None, 16, 16, 32)	4608	['block_3_depthwise_relu[0][0]']
block_3_project_BN (BatchNormalization)	(None, 16, 16, 32)	128	['block_3_project[0][0]']
block_4_expand (Conv2D)	(None, 16, 16, 192)	6144	['block_3_project_BN[0][0]']
'rm' is not recognized as an internal or external command, operable program or batch file.			
block_4_expand_BN (BatchNormalization)	(None, 16, 16, 192)	768	['block_4_expand[0][0]']
block_4_expand_relu (ReLU)	(None, 16, 16, 192)	0	['block_4_expand_BN[0][0]']
block_4_depthwise (DepthwiseConv2D)	(None, 16, 16, 192)	1728	['block_4_expand_relu[0][0]']
block_4_depthwise_BN (BatchNormalization)	(None, 16, 16, 192)	768	['block_4_depthwise[0][0]']
block_4_depthwise_relu (ReLU)	(None, 16, 16, 192)	0	['block_4_depthwise_BN[0][0]']
block_4_project (Conv2D)	(None, 16, 16, 32)	6144	['block_4_depthwise_relu[0][0]']
block_4_project_BN (BatchNormalization)	(None, 16, 16, 32)	128	['block_4_project[0][0]']
block_4_add (Add)	(None, 16, 16, 32)	0	['block_3_project_BN[0][0]', 'block_4_project_BN[0][0]']
block_5_expand (Conv2D)	(None, 16, 16, 192)	6144	['block_4_add[0][0]']

Basic Model Continuation3

block_5_expand_BN (BatchNormalization)	(None, 16, 16, 192)	768	['block_5_expand[0][0]']
block_5_expand_relu (ReLU)	(None, 16, 16, 192)	0	['block_5_expand_BN[0][0]']
block_5_depthwise (DepthwiseConv2D)	(None, 16, 16, 192)	1728	['block_5_expand_relu[0][0]']
block_5_depthwise_BN (BatchNormalization)	(None, 16, 16, 192)	768	['block_5_depthwise[0][0]']
block_5_depthwise_relu (ReLU)	(None, 16, 16, 192)	0	['block_5_depthwise_BN[0][0]']
block_5_project (Conv2D)	(None, 16, 16, 32)	6144	['block_5_depthwise_relu[0][0]']
block_5_project_BN (BatchNormalization)	(None, 16, 16, 32)	128	['block_5_project[0][0]']
block_5_add (Add)	(None, 16, 16, 32)	0	['block_4_add[0][0]', 'block_5_project_BN[0][0]']
block_6_expand (Conv2D)	(None, 16, 16, 192)	6144	['block_5_add[0][0]']
block_6_expand_BN (BatchNormalization)	(None, 16, 16, 192)	768	['block_6_expand[0][0]']
block_6_expand_relu (ReLU)	(None, 16, 16, 192)	0	['block_6_expand_BN[0][0]']
block_6_pad (ZeroPadding2D)	(None, 17, 17, 192)	0	['block_6_expand_relu[0][0]']
block_6_depthwise (DepthwiseConv2D)	(None, 8, 8, 192)	1728	['block_6_pad[0][0]']
block_6_depthwise_BN (BatchNormalization)	(None, 8, 8, 192)	768	['block_6_depthwise[0][0]']
block_6_depthwise_relu (ReLU)	(None, 8, 8, 192)	0	['block_6_depthwise_BN[0][0]']
block_6_project (Conv2D)	(None, 8, 8, 64)	12288	['block_6_depthwise_relu[0][0]']

Basic Model Continuation4

block_6_project_BN (BatchNormalization)	(None, 8, 8, 64)	256	['block_6_project[0][0]']
block_7_expand (Conv2D)	(None, 8, 8, 384)	24576	['block_6_project_BN[0][0]']
block_7_expand_BN (BatchNormalization)	(None, 8, 8, 384)	1536	['block_7_expand[0][0]']
block_7_expand_relu (ReLU)	(None, 8, 8, 384)	0	['block_7_expand_BN[0][0]']
block_7_depthwise (DepthwiseConv2D)	(None, 8, 8, 384)	3456	['block_7_expand_relu[0][0]']
block_7_depthwise_BN (BatchNormalization)	(None, 8, 8, 384)	1536	['block_7_depthwise[0][0]']
block_7_depthwise_relu (ReLU)	(None, 8, 8, 384)	0	['block_7_depthwise_BN[0][0]']
block_7_project (Conv2D)	(None, 8, 8, 64)	24576	['block_7_depthwise_relu[0][0]']
block_7_project_BN (BatchNormalization)	(None, 8, 8, 64)	256	['block_7_project[0][0]']
block_7_add (Add)	(None, 8, 8, 64)	0	['block_6_project_BN[0][0]', 'block_7_project_BN[0][0]']
block_8_expand (Conv2D)	(None, 8, 8, 384)	24576	['block_7_add[0][0]']
block_8_expand_BN (BatchNormalization)	(None, 8, 8, 384)	1536	['block_8_expand[0][0]']
block_8_expand_relu (ReLU)	(None, 8, 8, 384)	0	['block_8_expand_BN[0][0]']
block_8_depthwise (DepthwiseConv2D)	(None, 8, 8, 384)	3456	['block_8_expand_relu[0][0]']
block_8_depthwise_BN (BatchNormalization)	(None, 8, 8, 384)	1536	['block_8_depthwise[0][0]']
block_8_depthwise_relu (ReLU)	(None, 8, 8, 384)	0	['block_8_depthwise_BN[0][0]']

Basic Model Continuation5

block_8_project (Conv2D)	(None, 8, 8, 64)	24576	['block_8_depthwise_relu[0][0]']
block_8_project_BN (BatchNormalization)	(None, 8, 8, 64)	256	['block_8_project[0][0]']
block_8_add (Add)	(None, 8, 8, 64)	0	['block_7_add[0][0]', 'block_8_project_BN[0][0]']
block_9_expand (Conv2D)	(None, 8, 8, 384)	24576	['block_8_add[0][0]']
block_9_expand_BN (BatchNormalization)	(None, 8, 8, 384)	1536	['block_9_expand[0][0]']
block_9_expand_relu (ReLU)	(None, 8, 8, 384)	0	['block_9_expand_BN[0][0]']
block_9_depthwise (DepthwiseConv2D)	(None, 8, 8, 384)	3456	['block_9_expand_relu[0][0]']
block_9_depthwise_BN (BatchNormalization)	(None, 8, 8, 384)	1536	['block_9_depthwise[0][0]']
block_9_depthwise_relu (ReLU)	(None, 8, 8, 384)	0	['block_9_depthwise_BN[0][0]']
block_9_project (Conv2D)	(None, 8, 8, 64)	24576	['block_9_depthwise_relu[0][0]']
block_9_project_BN (BatchNormalization)	(None, 8, 8, 64)	256	['block_9_project[0][0]']
block_9_add (Add)	(None, 8, 8, 64)	0	['block_8_add[0][0]', 'block_9_project_BN[0][0]']
block_10_expand (Conv2D)	(None, 8, 8, 384)	24576	['block_9_add[0][0]']
block_10_expand_BN (BatchNormalization)	(None, 8, 8, 384)	1536	['block_10_expand[0][0]']
block_10_expand_relu (ReLU)	(None, 8, 8, 384)	0	['block_10_expand_BN[0][0]']
block_10_depthwise (DepthwiseConv2D)	(None, 8, 8, 384)	3456	['block_10_expand_relu[0][0]']

Basic Model Continuation6

block_10_depthwise_BN (BatchNormalization)	(None, 8, 8, 384)	1536	['block_10_depthwise[0][0]']
block_10_depthwise_relu (ReLU)	(None, 8, 8, 384)	0	['block_10_depthwise_BN[0][0]']
block_10_project (Conv2D)	(None, 8, 8, 96)	36864	['block_10_depthwise_relu[0][0]']
block_10_project_BN (BatchNormalization)	(None, 8, 8, 96)	384	['block_10_project[0][0]']
block_11_expand (Conv2D)	(None, 8, 8, 576)	55296	['block_10_project_BN[0][0]']
block_11_expand_BN (BatchNormalization)	(None, 8, 8, 576)	2304	['block_11_expand[0][0]']
block_11_expand_relu (ReLU)	(None, 8, 8, 576)	0	['block_11_expand_BN[0][0]']
block_11_depthwise (DepthwiseConv2D)	(None, 8, 8, 576)	5184	['block_11_expand_relu[0][0]']
block_11_depthwise_BN (BatchNormalization)	(None, 8, 8, 576)	2304	['block_11_depthwise[0][0]']
block_11_depthwise_relu (ReLU)	(None, 8, 8, 576)	0	['block_11_depthwise_BN[0][0]']
block_11_project (Conv2D)	(None, 8, 8, 96)	55296	['block_11_depthwise_relu[0][0]']
block_11_project_BN (BatchNormalization)	(None, 8, 8, 96)	384	['block_11_project[0][0]']
block_11_add (Add)	(None, 8, 8, 96)	0	['block_10_project_BN[0][0]', 'block_11_project_BN[0][0]']
block_12_expand (Conv2D)	(None, 8, 8, 576)	55296	['block_11_add[0][0]']
block_12_expand_BN (BatchNormalization)	(None, 8, 8, 576)	2304	['block_12_expand[0][0]']
block_12_expand_relu (ReLU)	(None, 8, 8, 576)	0	['block_12_expand_BN[0][0]']

Basic Model Continuation7

block_12_depthwise (DepthwiseConv2D)	(None, 8, 8, 576)	5184	['block_12_expand_relu[0][0]']
block_12_depthwise_BN (BatchNormalization)	(None, 8, 8, 576)	2304	['block_12_depthwise[0][0]']
block_12_depthwise_relu (ReLU)	(None, 8, 8, 576)	0	['block_12_depthwise_BN[0][0]']
block_12_project (Conv2D)	(None, 8, 8, 96)	55296	['block_12_depthwise_relu[0][0]']
block_12_project_BN (BatchNormalization)	(None, 8, 8, 96)	384	['block_12_project[0][0]']
block_12_add (Add)	(None, 8, 8, 96)	0	['block_11_add[0][0]', 'block_12_project_BN[0][0]']
block_13_expand (Conv2D)	(None, 8, 8, 576)	55296	['block_12_add[0][0]']
block_13_expand_BN (BatchNormalization)	(None, 8, 8, 576)	2304	['block_13_expand[0][0]']
block_13_expand_relu (ReLU)	(None, 8, 8, 576)	0	['block_13_expand_BN[0][0]']
block_13_pad (ZeroPadding2D)	(None, 9, 9, 576)	0	['block_13_expand_relu[0][0]']
block_13_depthwise (DepthwiseConv2D)	(None, 4, 4, 576)	5184	['block_13_pad[0][0]']
block_13_depthwise_BN (BatchNormalization)	(None, 4, 4, 576)	2304	['block_13_depthwise[0][0]']
block_13_depthwise_relu (ReLU)	(None, 4, 4, 576)	0	['block_13_depthwise_BN[0][0]']
block_13_project (Conv2D)	(None, 4, 4, 160)	92160	['block_13_depthwise_relu[0][0]']
block_13_project_BN (BatchNormalization)	(None, 4, 4, 160)	640	['block_13_project[0][0]']
block_14_expand (Conv2D)	(None, 4, 4, 960)	153600	['block_13_project_BN[0][0]']

Basic Model Continuation8

block_14_expand_BN (BatchNormalization)	(None, 4, 4, 960)	3840	['block_14_expand[0][0]']
block_14_expand_relu (ReLU)	(None, 4, 4, 960)	0	['block_14_expand_BN[0][0]']
block_14_depthwise (DepthwiseConv2D)	(None, 4, 4, 960)	8640	['block_14_expand_relu[0][0]']
block_14_depthwise_BN (BatchNormalization)	(None, 4, 4, 960)	3840	['block_14_depthwise[0][0]']
block_14_depthwise_relu (ReLU)	(None, 4, 4, 960)	0	['block_14_depthwise_BN[0][0]']
block_14_project (Conv2D)	(None, 4, 4, 160)	153600	['block_14_depthwise_relu[0][0]']
block_14_project_BN (BatchNormalization)	(None, 4, 4, 160)	640	['block_14_project[0][0]']
block_14_add (Add)	(None, 4, 4, 160)	0	['block_13_project_BN[0][0]', 'block_14_project_BN[0][0]']
block_15_expand (Conv2D)	(None, 4, 4, 960)	153600	['block_14_add[0][0]']
block_15_expand_BN (BatchNormalization)	(None, 4, 4, 960)	3840	['block_15_expand[0][0]']
block_15_expand_relu (ReLU)	(None, 4, 4, 960)	0	['block_15_expand_BN[0][0]']
block_15_depthwise (DepthwiseConv2D)	(None, 4, 4, 960)	8640	['block_15_expand_relu[0][0]']
block_15_depthwise_BN (BatchNormalization)	(None, 4, 4, 960)	3840	['block_15_depthwise[0][0]']
block_15_depthwise_relu (ReLU)	(None, 4, 4, 960)	0	['block_15_depthwise_BN[0][0]']
block_15_project (Conv2D)	(None, 4, 4, 160)	153600	['block_15_depthwise_relu[0][0]']

Basic Model Continuation9

block_15_project_BN (BatchNormalization)	(None, 4, 4, 160)	640	['block_15_project[0][0]']
block_15_add (Add)	(None, 4, 4, 160)	0	['block_14_add[0][0]', 'block_15_project_BN[0][0]']
block_16_expand (Conv2D)	(None, 4, 4, 960)	153600	['block_15_add[0][0]']
block_16_expand_BN (BatchNormalization)	(None, 4, 4, 960)	3840	['block_16_expand[0][0]']
block_16_expand_relu (ReLU)	(None, 4, 4, 960)	0	['block_16_expand_BN[0][0]']
block_16_depthwise (DepthwiseConv2D)	(None, 4, 4, 960)	8640	['block_16_expand_relu[0][0]']
block_16_depthwise_BN (BatchNormalization)	(None, 4, 4, 960)	3840	['block_16_depthwise[0][0]']
block_16_depthwise_relu (ReLU)	(None, 4, 4, 960)	0	['block_16_depthwise_BN[0][0]']
block_16_project (Conv2D)	(None, 4, 4, 320)	307200	['block_16_depthwise_relu[0][0]']
block_16_project_BN (BatchNormalization)	(None, 4, 4, 320)	1280	['block_16_project[0][0]']
Conv_1 (Conv2D)	(None, 4, 4, 1280)	409600	['block_16_project_BN[0][0]']
Conv_1_bn (BatchNormalization)	(None, 4, 4, 1280)	5120	['Conv_1[0][0]']
out_relu (ReLU)	(None, 4, 4, 1280)	0	['Conv_1_bn[0][0]']
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0	['out_relu[0][0]']
dense (Dense)	(None, 1)	1281	['global_average_pooling2d[0][0]']

5.1.1.4 Parameters that can be trained

```

=====
Total params: 2,259,265
Trainable params: 1,281
Non-trainable params: 2,257,984

```

Number of parameters that can be trained from the basic model based on the conditions of the required model.

5.1.1.5 Training of model

```

Training head...
Epoch 1/25

 1/2439 [.....] - ETA: 1:09:12 - loss: 1.4405 - accuracy: 0.2500
 3/2439 [.....] - ETA: 1:16 - loss: 1.2597 - accuracy: 0.4167
 6/2439 [.....] - ETA: 1:08 - loss: 1.2310 - accuracy: 0.3750
 9/2439 [.....] - ETA: 1:01 - loss: 1.3720 - accuracy: 0.3889
12/2439 [.....] - ETA: 1:02 - loss: 1.2103 - accuracy: 0.4375
15/2439 [.....] - ETA: 59s - loss: 1.2312 - accuracy: 0.4000
18/2439 [.....] - ETA: 1:00 - loss: 1.1903 - accuracy: 0.4167
21/2439 [.....] - ETA: 58s - loss: 1.0965 - accuracy: 0.4524
24/2439 [.....] - ETA: 59s - loss: 1.1457 - accuracy: 0.4479
27/2439 [.....] - ETA: 58s - loss: 1.1510 - accuracy: 0.4537
30/2439 [.....] - ETA: 57s - loss: 1.1344 - accuracy: 0.4417
32/2439 [.....] - ETA: 57s - loss: 1.1239 - accuracy: 0.4453
35/2439 [.....] - ETA: 57s - loss: 1.0824 - accuracy: 0.4643
38/2439 [.....] - ETA: 57s - loss: 1.0789 - accuracy: 0.4671
41/2439 [.....] - ETA: 57s - loss: 1.0723 - accuracy: 0.4634
43/2439 [.....] - ETA: 57s - loss: 1.0955 - accuracy: 0.4477
46/2439 [.....] - ETA: 58s - loss: 1.1226 - accuracy: 0.4402
49/2439 [.....] - ETA: 58s - loss: 1.1267 - accuracy: 0.4337
52/2439 [.....] - ETA: 57s - loss: 1.1266 - accuracy: 0.4327
55/2439 [.....] - ETA: 58s - loss: 1.1151 - accuracy: 0.4409
58/2439 [.....] - ETA: 57s - loss: 1.1429 - accuracy: 0.4353
61/2439 [.....] - ETA: 57s - loss: 1.1505 - accuracy: 0.4426
64/2439 [.....] - ETA: 57s - loss: 1.1693 - accuracy: 0.4336
67/2439 [.....] - ETA: 56s - loss: 1.1482 - accuracy: 0.4440

```

Training the model

```

2433/2439 [=====>.] - ETA: 0s - loss: 0.8890 - accuracy: 0.5091
2436/2439 [=====>.] - ETA: 0s - loss: 0.8888 - accuracy: 0.5092
2439/2439 [=====] - ETA: 0s - loss: 0.8885 - accuracy: 0.5095
Epoch 1: val_loss improved from inf to 0.74558, saving model to ModelWeights.h5

2439/2439 [=====] - 94s 38ms/step - loss: 0.8885 - accuracy: 0.5095 - val_loss: 0.7456 - val_accuracy: 0.5956 - lr: 1.0000e-05
Epoch 2/25

 1/2439 [.....] - ETA: 1:23 - loss: 0.5305 - accuracy: 0.7500
 4/2439 [.....] - ETA: 54s - loss: 0.4883 - accuracy: 0.7500
 6/2439 [.....] - ETA: 56s - loss: 0.5612 - accuracy: 0.6667
 9/2439 [.....] - ETA: 55s - loss: 0.6388 - accuracy: 0.6667
12/2439 [.....] - ETA: 55s - loss: 0.6805 - accuracy: 0.6458

```

loss, accuracy, val_loss and val_accuracy of epoch1 and starting of epoch2



M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)

```
2428/2439 [=====>.] - ETA: 0s - loss: 0.1251 - accuracy: 0.9568
2430/2439 [=====>.] - ETA: 0s - loss: 0.1250 - accuracy: 0.9568
2432/2439 [=====>.] - ETA: 0s - loss: 0.1252 - accuracy: 0.9566
2434/2439 [=====>.] - ETA: 0s - loss: 0.1252 - accuracy: 0.9567
2436/2439 [=====>.] - ETA: 0s - loss: 0.1251 - accuracy: 0.9567
2438/2439 [=====>.] - ETA: 0s - loss: 0.1251 - accuracy: 0.9566
Epoch 14: val_loss improved from 0.14154 to 0.14063, saving model to ModelWeights.h5

2439/2439 [=====] - 91s 37ms/step - loss: 0.1251 - accuracy: 0.9566 - val_loss: 0.1406 - val_accuracy: 0.9495 - lr: 1.6711e-05
Epoch 15/25

1/2439 [.....] - ETA: 1:22 - loss: 0.0256 - accuracy: 1.0000
3/2439 [.....] - ETA: 1:05 - loss: 0.1580 - accuracy: 0.9167
6/2439 [.....] - ETA: 1:01 - loss: 0.1403 - accuracy: 0.9167
9/2439 [.....] - ETA: 1:03 - loss: 0.1488 - accuracy: 0.9167
11/2439 [.....] - ETA: 1:05 - loss: 0.1629 - accuracy: 0.9091
14/2439 [.....] - ETA: 1:01 - loss: 0.1904 - accuracy: 0.9107
17/2439 [.....] - ETA: 58s - loss: 0.1926 - accuracy: 0.9118
20/2439 [.....] - ETA: 58s - loss: 0.1778 - accuracy: 0.9250
```

loss, accuracy, val_loss and val_accuracy of epoch14 and starting of epoch15

```
2412/2439 [=====>.] - ETA: 0s - loss: 0.1092 - accuracy: 0.9633
2414/2439 [=====>.] - ETA: 0s - loss: 0.1091 - accuracy: 0.9633
2416/2439 [=====>.] - ETA: 0s - loss: 0.1091 - accuracy: 0.9634
2418/2439 [=====>.] - ETA: 0s - loss: 0.1091 - accuracy: 0.9634
2420/2439 [=====>.] - ETA: 0s - loss: 0.1091 - accuracy: 0.9634
2422/2439 [=====>.] - ETA: 0s - loss: 0.1095 - accuracy: 0.9632
2424/2439 [=====>.] - ETA: 0s - loss: 0.1094 - accuracy: 0.9632
2426/2439 [=====>.] - ETA: 0s - loss: 0.1097 - accuracy: 0.9631
2428/2439 [=====>.] - ETA: 0s - loss: 0.1096 - accuracy: 0.9631
2430/2439 [=====>.] - ETA: 0s - loss: 0.1096 - accuracy: 0.9632
2432/2439 [=====>.] - ETA: 0s - loss: 0.1098 - accuracy: 0.9631
2434/2439 [=====>.] - ETA: 0s - loss: 0.1097 - accuracy: 0.9631
2436/2439 [=====>.] - ETA: 0s - loss: 0.1097 - accuracy: 0.9632
2438/2439 [=====>.] - ETA: 0s - loss: 0.1098 - accuracy: 0.9631
Epoch 25: val_loss improved from 0.12948 to 0.12884, saving model to ModelWeights.h5

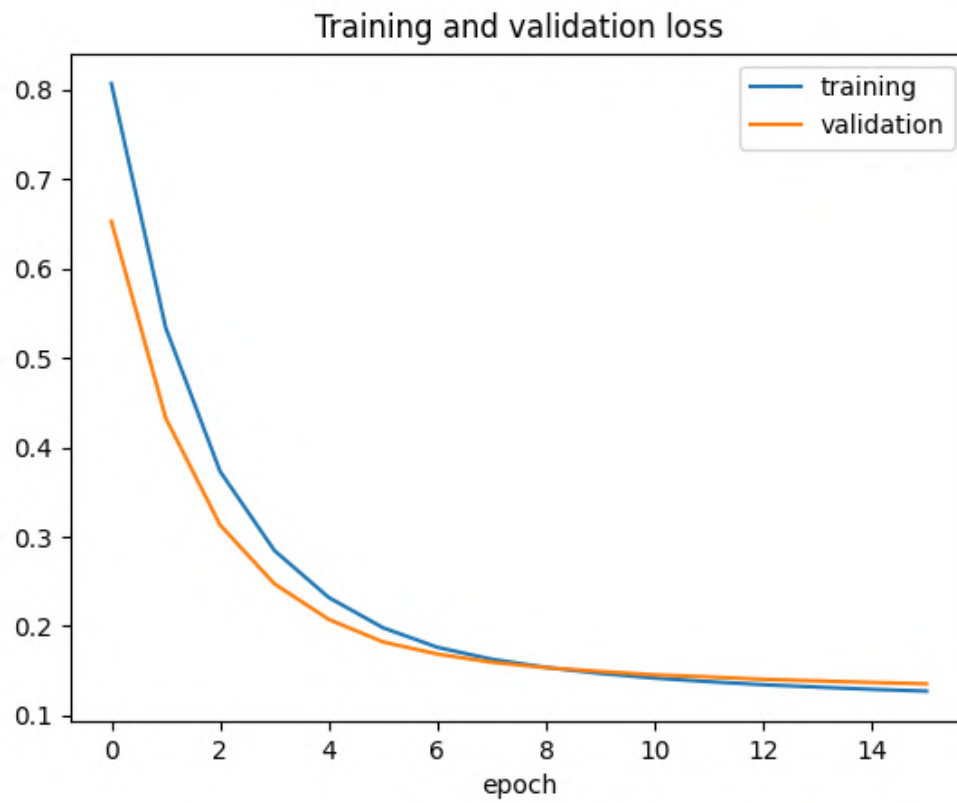
2439/2439 [=====] - 121s 50ms/step - loss: 0.1098 - accuracy: 0.9631 - val_loss: 0.1288 - val_accuracy: 0.9536 - lr: 1.0576e-05
```

loss, accuracy, val_loss and val_accuracy of epoch25

```
Restoring best Weights for MobileNetV2
-----
Best Epochs: 17
Accuracy on train: 0.960225522518158    Loss on train: 0.11777835339307785
Accuracy on test: 0.9536107182502747    Loss on test: 0.1288442462682724
-----
```

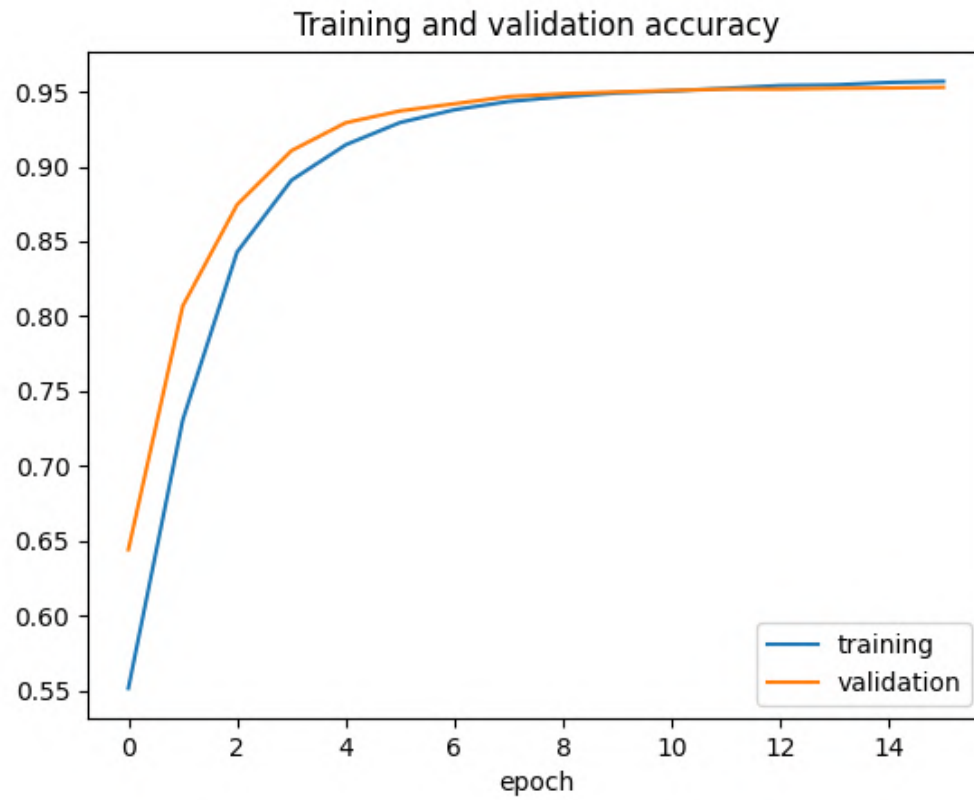
Predicting the best epochs, calculating the accuracy of training and testing data and loss
of training and testing data

5.1.1.6 Training and validation loss



Graph for training and validation loss

5.1.1.7 Training and validation accuracy



Graph for Training and validation accuracy

5.1.1.8 Evaluation of network:

```
Evaluating network...

1/131 [.....] - ETA: 2:45
2/131 [.....] - ETA: 18s
3/131 [.....] - ETA: 19s
4/131 [.....] - ETA: 19s
5/131 [>.....] - ETA: 18s
6/131 [>.....] - ETA: 18s
7/131 [>.....] - ETA: 18s
8/131 [>.....] - ETA: 18s
9/131 [=>.....] - ETA: 18s
10/131 [=>.....] - ETA: 18s
11/131 [=>.....] - ETA: 18s
12/131 [=>.....] - ETA: 18s
13/131 [=>.....] - ETA: 18s
14/131 [==>.....] - ETA: 19s
15/131 [==>.....] - ETA: 19s
16/131 [==>.....] - ETA: 18s
17/131 [==>.....] - ETA: 18s
18/131 [===>.....] - ETA: 18s
19/131 [===>.....] - ETA: 18s
20/131 [===>.....] - ETA: 18s
21/131 [===>.....] - ETA: 18s
22/131 [====>.....] - ETA: 18s
23/131 [====>.....] - ETA: 18s
24/131 [====>.....] - ETA: 18s
25/131 [====>.....] - ETA: 18s
26/131 [====>.....] - ETA: 17s
27/131 [====>.....] - ETA: 17s
28/131 [====>.....] - ETA: 17s
```

Evaluation of network

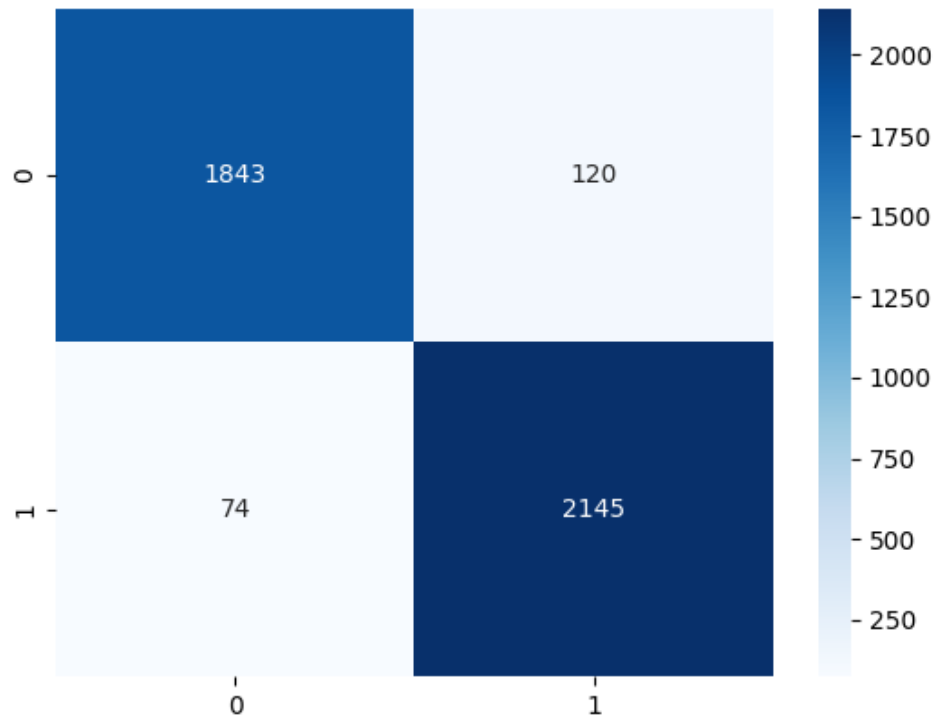
```
> Correct Predictions: 3988
> Wrong Predictions: 194

      precision    recall  f1-score   support

NonViolence      0.96      0.94      0.95      1963
Violence          0.95      0.97      0.96      2219

 accuracy          0.95          0.95          0.95      4182
 macro avg         0.95          0.95          0.95      4182
weighted avg         0.95          0.95          0.95      4182
```

Correct and wrong predictions based model evaluation and corresponding confusion matrix.



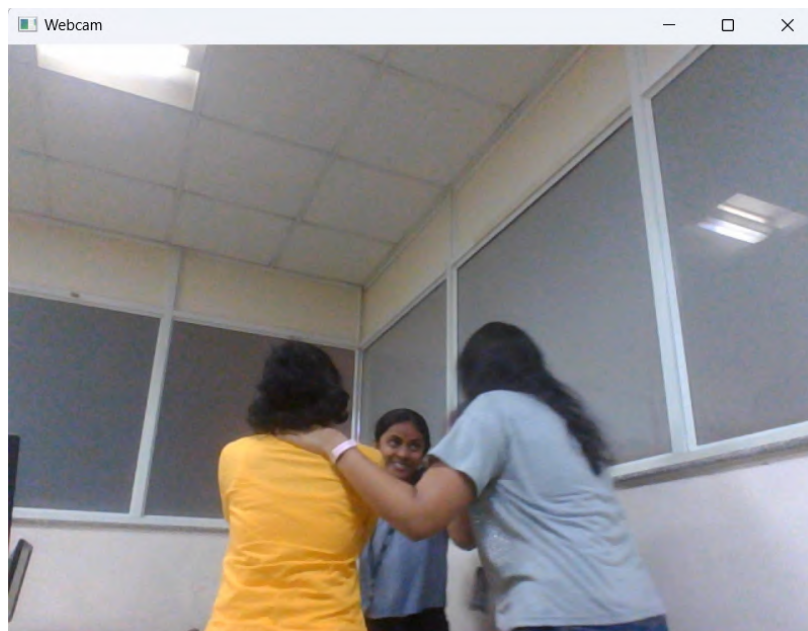
Graphical plot for Confusion matrix of model evaluation

5.1.2 Case 1: Violent actions detected

5.1.2.1 Video Capture:



Screenshot while video capturing 1



Screenshot while video capturing 2

5.1.2.2 Face Detection:



Faces saved when face recognition algorithm was applied

5.1.2.3 Action Detection with Labels:

```

1/1 [=====] - ETA: 0s
1/1 [=====] - 1s 819ms/step
1

1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 40ms/step
2

1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 43ms/step
3

1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 30ms/step
4

1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 29ms/step
5

1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 34ms/step
6

1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 29ms/step
7

1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 36ms/step
8

```

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)
Prediction process for every frame in the video



Violence detection with label 1



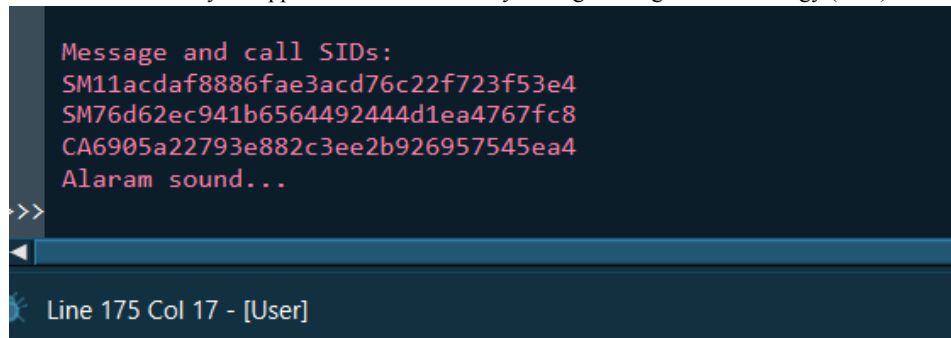
Violence detection with label 2

5.1.2.4 Notifications:

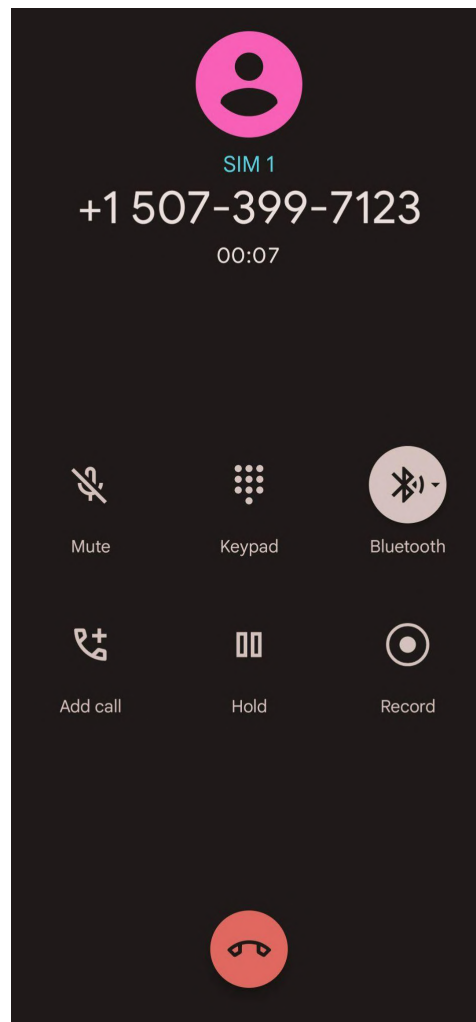
```
Violent

GetLocation() starts...
[13.097327, 77.409626]
373, Shree Gurukrupa nilaya, 1st cross, Nelamangala new town layout , Nelamangala taluk, Jyothi Nagar, Karnataka 562123, India
https://google.com/search?q=13.097327,77.409626
GetLocation() ends...
```

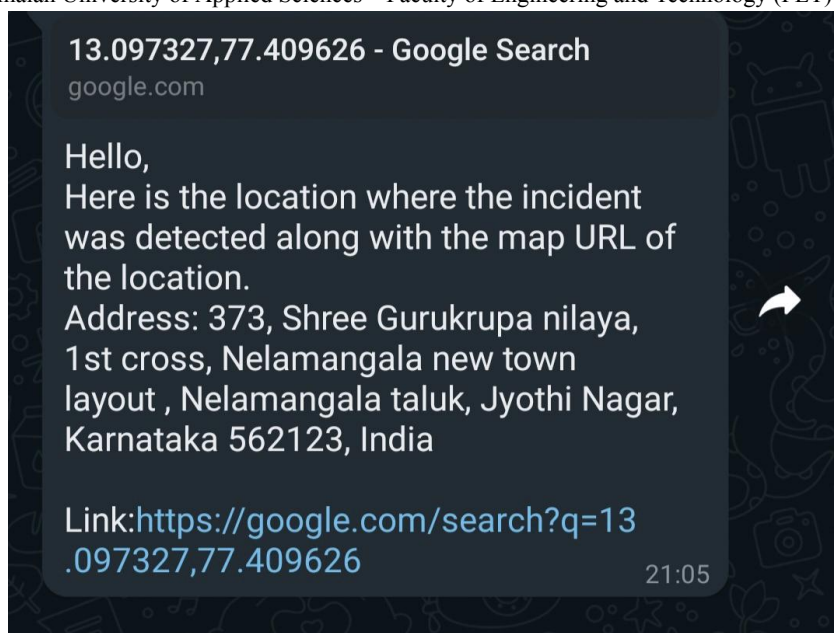
Violence identified and the location is fetched



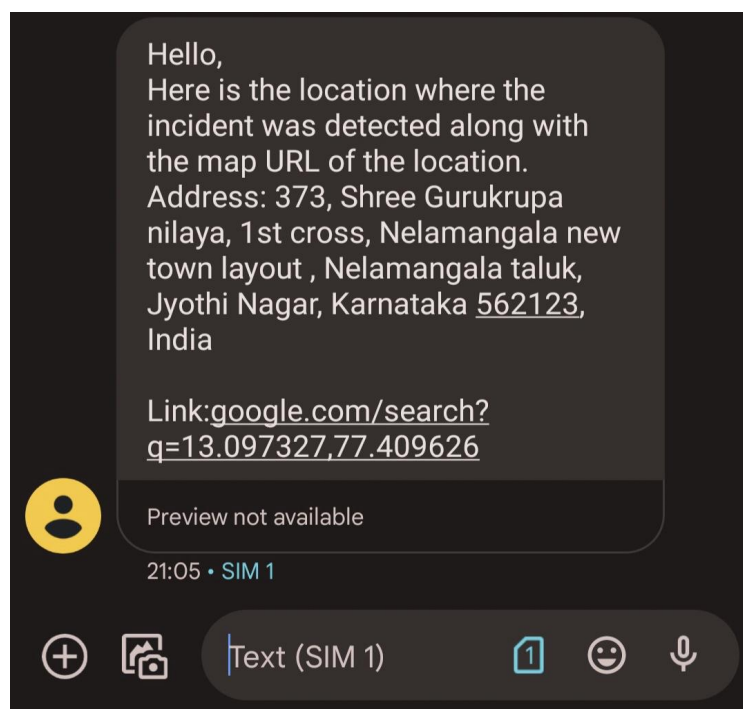
Call and messages are sent with location when violence is detected and alarm is triggered.



Call received on the recipient's phone



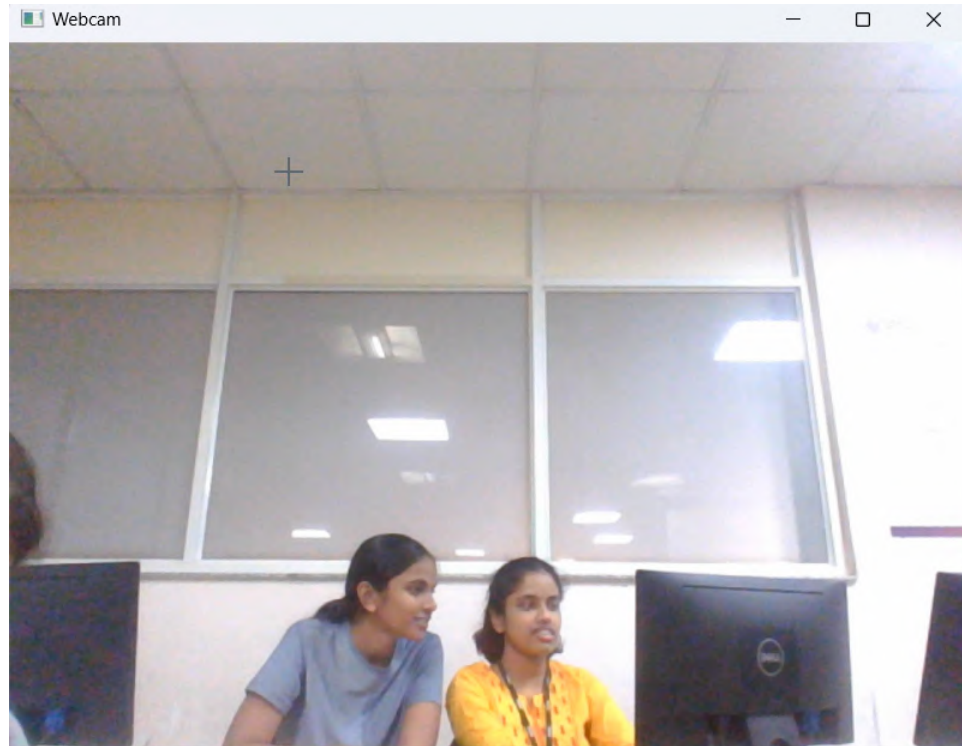
Whatsapp message



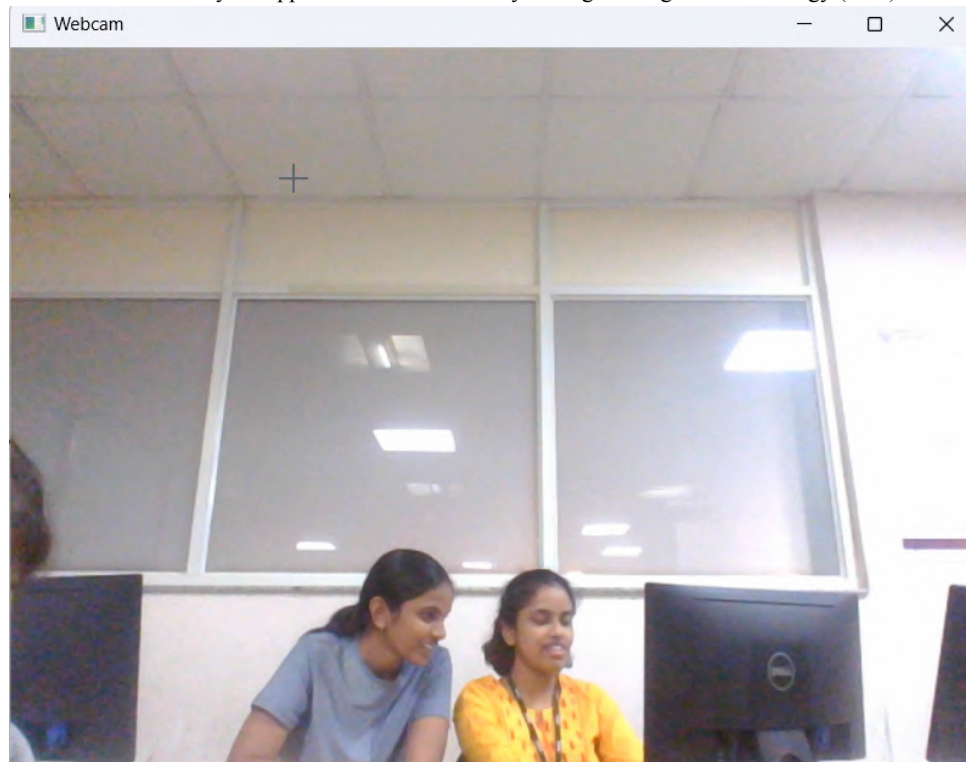
Text Message

5.1.3 Case 2: Non Violent actions detected

5.1.3.1 Video Capture:



Screenshot while video capturing 1



Screenshot while video capturing 2

5.1.3.2 Face Detection:



Faces saved when face recognition algorithm was applied

5.1.3.3 Action Detection with Labels:



Non Violence detection with label

5.1.3.4 Notification:

```
1/1 [=====] - 0s 32ms/step
1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 32ms/step
0
Non-Violent
```

The actions are non violent. No notifications are sent.

5.2 Summary

The model is trained using both violent and non violent datasets and the corresponding trained and validation loss and training and validation accuracy graphs are plotted. The trained model is evaluated for network to check how many predictions are correct and wrong predictions are made and a corresponding confusion matrix is plotted.

The webcam is accessed to capture the real time video. The face detection algorithm is applied to the video to obtain the faces and stores them in the specified path. After face recognition, action recognition model is applied to the video. If the actions in the video are predicted to be violent, notifications(call and messages) are sent along with the location where the violence occurred. If the actions in the video are predicted to be non violent then no notifications are sent.

6. Project Costing

This chapter deals with costing of this project which gives an overall estimation of expenses that was required to complete this project. This Covers expenses of testing devices, Platform and Hardware cost, Human Resource Cost and a grand total of Entire cost.

6.1 Project Cost Estimation

The cost of project is summarised in a tabular form displayed below:

Table 5 Cost estimation table

Serial Number	Resources and Work	Cost(Rs)
1	Laptop (for development)	45,000/-
2	Twilio software	1640/-
3	Human Resources (5 * 5,000/-)	25,000/-
	TOTAL	71,640/-

6.2 Conclusion

The system was designed in such a manner so as to reduce costs as much as possible. The paid version of Twilio software was chosen in order to make the system less dependent on human intervention since the trial version of Twilio required the user to authenticate their phone number with Twilio before receiving calls or messages from the Twilio number. However, effort on training and testing of the AI model via parallel learning of new technology raised the Human Resource cost.

The dataset was trained and tested on a laptop without a GPU, and real-time video of the environment was recorded using the laptop's built-in camera.

7. Conclusions and Suggestions for Future Work

In this chapter, conclusion has been given for entire project along with conclusion to each section present in the report. All are explained with status of completion of each section mentioned clearly. This section ends with Suggestion and scope of future work which direct this project towards new openings of technology where the same project can be extended in order to meet the requirement of customers time to time.

7.1 Conclusion

This project started with a Literature survey done via gathering information from different IEEE papers, patented documents and reputed Websites. Books from Orally publication were also used as a guide. Also, popular applications were listed against their features making it clearer to compare and contrast applications with each other along with algorithms proposed in this project. Background Theory of all resources including technology used, Framework selected, IDE's worked upon and the modules applied were extensively elaborated so that these theories can be applied effectively and the reason for their use/ application can be well understood. Later, all objectives were listed after declaring title and Aim of the project i.e "Detection of Violent Crimes" and methods and methodologies in order to complete the bulleted objectives were well tabulated. From objectives, Functional Requirements were extracted and were well segregated for sequential completion of project. Implementation of the project was displayed via displaying code written in Python programming language using the modules of Artificial intelligence and Neural Network interacting with each other to create the application. Testing was done for all functionalities and were found to be working successfully. Later in the result section, all screenshots of results in different states were taken in order to demonstrate the objective and usefulness of this project. Each screenshot was explained with its importance as a view for the application. Performance analysis was done in order to give numerical value to performance of the application clearly stating the

M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)
advancement in application proposed in this project compared to other applications present in the market. Later project cost estimation was done in order to know the financial asset required to rebuild this project. The entire project was concluded with a suitable conclusion and its scope in near future.

7.2 Suggestion for future work

Innovation is a never ending process, hence bring new innovation and extension of this project is always possible. This project for now is limited to 3 functionalities, namely:

- Face detection: The developed system can detect and locally store pictures of, faces of people in the video.

This functionality can be further improved by integrating face detection with face recognition, which involves implementing a model which is trained using an existing dataset which should contain the images and details of active and passive habitual perpetrators, to recognize and control the damage so as to prevent the crime from escalating.

- Violent action detection: The action detection model is trained and designed to only detect whether the action detected is violent or not. This functionality can be further improved by increasing the size of datasets used for training and testing can be increased to get different kinds of violent actions that might not be present in the current dataset. The efficiency of the model can be increased by increasing the number of epochs to train the model.

In the current detection method, we are having few wrong predictions. To solve this we can use a much more efficient model that has much accuracy of exactly predicting whether the violence has occurred along with labelling what kind of violent action is committed.

- Location access and notification of incident: Location where the violent activity was detected is determined using geolocator. SMS is sent directly to the

recipient's phone number that was entered in the code, which is the same case for making phone calls.

This functionality can be improved by dynamically accessing the phone number of the nearby police stations.

In future phases of the project, the appropriate UI can be prepared so as to make the system more appealing and friendly to users. Developing an app or a website to not only allow the authorities to personally monitor the hotspots for crimes, but also to obtain messages and other notifications on a single platform so as to make the entire notification process less confusing and more trusting of the source of the notifications.

1. Bellotti, V., Dalal, B., Good, N., Flynn, P., Bobrow, D.G. and Ducheneaut, N., 2004, April. What a to-do: studies of task management towards the design of a personal task list manager. In *Proceedings of the SIGCHI conference on Human factors in computing systems* (pp. 735-742). ACM.
2. Hufnagel, M.R., Lacock, T.S. and Shook, A.K., International Business Machines Corp, 2012. *Auto-generated to-do list*. U.S. Patent 8,108,206.
3. Jenson, S.A., Apple Inc, 1996. *Schedule and to-do list for a pen-based computer system*. U.S. Patent 5,570,109.
4. Parekh, H. and Delvadiya, D., 2016. TO Do List Application.
5. Aarabi, P., & Moeini, A. (2019). Deep learning face recognition and its ethical considerations in public surveillance. *Journal of Big Data*, 6(1), 1-17.
6. Benitez-Rangel, J. P., & Ochoa, S. F. (2021). Review of facial recognition systems based on deep learning for security and surveillance applications. *Journal of Ambient Intelligence and Humanized Computing*, 1-27.
7. Chen, L., Wang, S., & Li, X. (2020). Crime prediction using machine learning techniques: A systematic literature review. *Computers & Security*, 92, 101682.
8. Hjelm, R. D., & Hu, J. (2020). Face recognition for surveillance purposes: A critical review. *Journal of Privacy and Confidentiality*, 10(2), 1-19.
9. Kopp, M., Maritzak, M., & Steinmetz, R. (2020). Social acceptance of face recognition technology: A systematic review and meta-analysis. *PloS one*, 15(8), e0237916.
10. Singh, A., & Vatsa, M. (2019). Action recognition for surveillance applications: A review. *Pattern Recognition Letters*, 125, 291-300.
11. Valerio, G., Gualdrini, G., & Nesi, P. (2019). Video-surveillance for public safety: A review. *Computer Science Review*, 34, 100187.



M.S.Ramaiah University of Applied Sciences – Faculty of Engineering and Technology (FET)

12. · Wang, C., Cao, Y., Guo, Y., Hu, X., & Zhang, J. (2020). Recent advances in video-based action recognition: A review. *Journal of Visual Communication and Image Representation*, 72, 102020.

Appendix

Dataset:

Access the dataset using the below link:

<https://www.kaggle.com/datasets/mohamedmustafa/real-life-violence-situations-dataset>

Code Snippets for Action Recognition Model Training:

```
video_to_frames
1 import os
2 import platform
3 from IPython.display import clear_output
4 import seaborn as sns
5 from sklearn import metrics
6 from sklearn.metrics import roc_curve, roc_auc_score, RocCurveDisplay, accuracy_score, classification_report, confusion_matrix
7 import cv2
8 import datetime
9 import imageio
10 import imgaug.augmenters as iaa
11 import imgaug as ia
12 import tensorflow as tf
13 import math
14 from tqdm import tqdm
15 import numpy as np
16 import pickle
17 import matplotlib
18 from sklearn.model_selection import StratifiedShuffleSplit
19 from keras.layers import Input
20 from keras.models import Model
21 from keras.layers.core import Dropout, Flatten, Dense
22 import matplotlib.pyplot as plt
23 from keras import regularizers
24 from keras.applications.mobilenet_v2 import MobileNetV2
25 from tensorflow.keras.callbacks import Callback, ModelCheckpoint, LearningRateScheduler, TensorBoard, EarlyStopping, ReduceLROnPlateau
26
```

Snippet 1

```
27 tf.random.set_seed(73)
28 TPU_INIT = False
29
30 if TPU_INIT:
31     try:
32         tpu = tf.distribute.cluster_resolver.TPUClusterResolver.connect()
33         tpu_strategy = tf.distribute.experimental.TPUStrategy(tpu)
34     except ValueError:
35         raise BaseException('ERROR: Not connected to a TPU runtime!')
36 else:
37     print('!nvidia-smi')
38     print("Tensorflow version " + tf.__version__)
39
40 IMG_SIZE = 128
41 ColorChannels = 3
```

Snippet 2

```
44 = def video_to_frames(video):
45     vidcap = cv2.VideoCapture(video)
46
47
48     rate = math.floor(vidcap.get(3))
49     count = 0
50
51     ImageFrames = []
52 =     while vidcap.isOpened():
53         ID = vidcap.get(1)
54         success, image = vidcap.read()
55
56 =         if success:
57             # skipping frames to avoid duplications
58 =             if (ID % 7 == 0):
59                 flip = iaa.Fliplr(1.0)
60                 zoom = iaa.Affine(scale=1.3)
61                 random_brightness = iaa.Multiply((1, 1.3))
62                 rotate = iaa.Affine(rotate=(-25, 25))
63
64                 image_aug = flip(image = image)
65                 image_aug = random_brightness(image = image_aug)
66                 image_aug = zoom(image = image_aug)
67                 image_aug = rotate(image = image_aug)
68
69                 rgb_img = cv2.cvtColor(image_aug, cv2.COLOR_BGR2RGB)
70                 resized = cv2.resize(rgb_img, (IMG_SIZE, IMG_SIZE))
71                 ImageFrames.append(resized)
72
73                 count += 1
74 =             else:
75                 break
76
77     vidcap.release()
78
79     return ImageFrames
80
```

Snippet 3

```

82
83     VideoDataDir = r"C:\Users\Varshitha V\Videos\Main Dataset"
84
85
86     X_original = []
87     y_original = []
88
89
90     CLASSES = ["NonViolence", "Violence"]
91
92
93     for category in os.listdir(VideoDataDir):
94         path = os.path.join(VideoDataDir, category)
95         class_num = CLASSES.index(category)
96         for i, video in enumerate(tqdm(os.listdir(path)[0:350])):
97             frames = video_to_frames(path + '/' + video)
98             for j, frame in enumerate(frames):
99                 X_original.append(frame)
100                 y_original.append(class_num)
101
102
103     X_original = np.array(X_original).reshape(-1, IMG_SIZE * IMG_SIZE * 3)
104     y_original = np.array(y_original)
105     len(X_original)
106
107
108     X_original = np.array(X_original).reshape(-1, IMG_SIZE * IMG_SIZE * 3)
109     y_original = np.array(y_original)
110     len(X_original)
111
112
113
114     stratified_sample = StratifiedShuffleSplit(n_splits=2, test_size=0.3, random_state=73)
115
116     for train_index, test_index in stratified_sample.split(X_original, y_original):
117         X_train, X_test = X_original[train_index], X_original[test_index]
118         y_train, y_test = y_original[train_index], y_original[test_index]
119
120     X_train_nn = X_train.reshape(-1, IMG_SIZE, IMG_SIZE, 3) / 255
121     X_test_nn = X_test.reshape(-1, IMG_SIZE, IMG_SIZE, 3) / 255
122
123     clear_output()
124

```

Snippet 4

```

127     epochs = 25
128
129
130     kernel_regularizer = regularizers.l2(0.0001)
131
132
133
134     def load_layers():
135         input_tensor = Input(shape=(IMG_SIZE, IMG_SIZE, ColorChannels))
136         baseModel = MobileNetV2(pooling='avg',
137                                 include_top=False,
138                                 input_tensor=input_tensor)
139
140         headModel = baseModel.output
141         headModel = Dense(1, activation="sigmoid")(headModel)
142         model = Model(inputs=baseModel.input, outputs=headModel)
143
144         for layer in baseModel.layers:
145             layer.trainable = False
146
147         print("Compiling model...")
148         model.compile(loss="binary_crossentropy",
149                     optimizer='adam',
150                     metrics=["accuracy"])
151
152         return model
153
154     if TPU_INIT:
155         with tpu_strategy.scope():
156             model = load_layers()
157     else:
158         model = load_layers()
159
160     model.summary()

```

Snippet 5

```

164     patience = 3
165
166     start_lr = 0.00001
167     min_lr = 0.00001
168     max_lr = 0.00005
169
170     batch_size = 4
171
172     if TPU_INIT:
173         max_lr = max_lr * tpu_strategy.num_replicas_in_sync
174         batch_size = batch_size * tpu_strategy.num_replicas_in_sync
175
176     rampup_epochs = 5
177     sustain_epochs = 0
178     exp_decay = .8
179
180     def lr_fn(epoch):
181         if epoch < rampup_epochs:
182             return (max_lr - start_lr)/rampup_epochs * epoch + start_lr
183         elif epoch < rampup_epochs + sustain_epochs:
184             return max_lr
185         else:
186             return (max_lr - min_lr) * exp_decay**(epoch-rampup_epochs-sustain_epochs) + min_lr
187
188
189     class myCallback(Callback):
190         def on_epoch_end(self, epoch, logs={}):
191             if ((logs.get('accuracy')>=0.999)):
192                 print("\nLimits Reached cancelling training!")
193                 self.model.stop_training = True
194     end_callback = myCallback()
195
196     lr_callback = LearningRateScheduler(lambda epoch: lr_fn(epoch), verbose=False)
197
198     early_stopping = EarlyStopping(patience = patience, monitor='val_loss',
199                                   mode='min', restore_best_weights=True,
200                                   verbose = 1, min_delta = .00075)
201

```

Snippet 6

```

201
202 PROJECT_DIR =r"C:\Users\Varshitha V\Videos\Main Dataset"
203
204 lr_plat = ReduceLROnPlateau(patience = 2, mode = 'min')
205
206 os.system('rm -rf ./logs/')
207
208
209 log_dir="logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
210 tensorboard_callback = TensorBoard(log_dir = log_dir, write_graph=True, histogram_freq=1)
211
212 checkpoint_filepath = 'ModelWeights.h5'
213
214 = model_checkpoints = ModelCheckpoint(filepath=checkpoint_filepath,
215 =                               save_weights_only=True,
216                               monitor='val_loss',
217                               mode='min',
218                               verbose = 1,
219                               save_best_only=True)
220
221
222 callbacks = [end_callback, lr_callback, model_checkpoints, tensorboard_callback, early_stopping, lr_plat]
223
224 = if TPU_INIT:
225     callbacks = [end_callback, lr_callback, model_checkpoints, early_stopping, lr_plat]
226
227 print('Training head...')
228 #model.load_weights('./Model_Weights.h5')
229
230 = history = model.fit(X_train_nn ,y_train, epochs=epochs,
231 =                   callbacks=callbacks,
232                   validation_data = (X_test_nn, y_test),
233                   batch_size=batch_size)
234
235 print('\nRestoring best Weights for MobileNetV2')
236 model.load_weights(checkpoint_filepath)
237

```

Snippet 7

```

239
240 = def print_graph(item, index, history):
241     plt.figure()
242     train_values = history.history[item][0:index]
243     plt.plot(train_values)
244     test_values = history.history['val_' + item][0:index]
245     plt.plot(test_values)
246     plt.legend(['training', 'validation'])
247     plt.title('Training and validation ' + item)
248     plt.xlabel('epoch')
249     plt.show()
250     plot = '{}.png'.format(item)
251     plt.savefig(plot)
252
253
254 = def get_best_epoch(test_loss, history):
255 =     for key, item in enumerate(history.history.items()):
256         (name, arr) = item
257         if name == 'val_loss':
258             for i in range(len(arr)):
259                 if round(test_loss, 2) == round(arr[i], 2):
260                     return i
261
262 = def model_summary(model, history):
263     print('---'*30)
264     test_loss, test_accuracy = model.evaluate(X_test_nn, y_test, verbose=0)
265
266 =     if history:
267         index = get_best_epoch(test_loss, history)
268         print('Best Epochs: ', index)
269
270         train_accuracy = history.history['accuracy'][index]
271         train_loss = history.history['loss'][index]
272
273         print('Accuracy on train:', train_accuracy, '\tLoss on train:', train_loss)
274         print('Accuracy on test:', test_accuracy, '\tLoss on test:', test_loss)
275         print_graph('loss', index, history)
276         print_graph('accuracy', index, history)
277         print('---'*30)
278
279
280     model_summary(model, history)

```

Snippet 8


```

282 # evaluate the network
283 print("Evaluating network...")
284 predictions = model.predict(X_test_nn)
285 preds = predictions > 0.5
286
287
288 corr_pred = metrics.confusion_matrix(y_test, preds)
289
290 n_correct = +int((corr_pred[0][0] + corr_pred[1][1]))
291 print('> Correct Predictions:', n_correct)
292 n_wrongs = int((corr_pred[0][1] + (corr_pred[1][0])))
293 print('> Wrong Predictions:', n_wrongs)
294
295 sns.heatmap(corr_pred,annot=True, fmt="d",cmap="Blues")
296 plt.show()
297
298 print(metrics.classification_report(y_test, preds,target_names=["NonViolence", "Violence"]))
299
300 args_model = "modelnew.h5"
301 model.save(args_model)

```

Snippet 9

Code Snippets for Web accessing, Face Recognition, Action Recognition, Notification and Alarm:

```

print_results (top)
1 import cv2
2 from cv2 import VideoWriter
3 from cv2 import VideoWriter_fourcc
4 from keras.models import load_model
5 from collections import deque
6 import matplotlib.pyplot as plt
7 import numpy as np
8 import argparse
9 import pickle
10 import os
11 import time
12 from keras.models import load_model
13 from collections import deque
14 from playsound import playsound
15 import os
16 from twilio.rest import Client
17 import winsdk.windows.devices.geolocation as wdg, asyncio
18 import requests
19 from requests import get
20 import googlemaps
21 import webbrowser
22

```

Import all the required libraries

```
23
24 # open the webcam video stream
25 webcam = cv2.VideoCapture(0)
26
27 # open output video file stream
28 video = VideoWriter('webcam.avi', VideoWriter_fourcc(*'MP42'), 25.0, (640, 480))
29 i=200
30 # main loop
31 = while (i>0):
32     # get the frame from the webcam
33     stream_ok, frame = webcam.read()
34
35     # if webcam stream is ok
36 =     if stream_ok:
37         # display current frame
38         cv2.imshow('Webcam', frame)
39
40         # write frame to the video file
41         video.write(frame)
42         i=i-1
43
44     # escape condition
45     if cv2.waitKey(1) & 0xFF == 27: break
46
47 # clean ups
48 cv2.destroyAllWindows()
49
50 # release web camera stream
51 webcam.release()
52
53 # release video output file stream
54 video.release()
55
```

Webcam Accessing

```

56
57 = def print_results(video, limit=None):
58
59
60     haar_file = cv2.data.harcascades + 'haarcascade_frontalface_default.xml'
61
62     path1=r"C:\Users\Varshitha V\data"
63     # defining the size of images
64     (width, height) = (130, 100)
65     vs = cv2.VideoCapture(video)
66     #'0' is used for my webcam,
67     # if you've any other camera
68     # attached use '1' like this
69     face_cascade = cv2.CascadeClassifier(haar_file)
70
71     # The program loops until it has 30 images of the face.
72     count = 1
73 = while count < 100:
74     (_, im) = vs.read()
75     gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
76     faces = face_cascade.detectMultiScale(gray, 1.3, 4)
77 =     for (x, y, w, h) in faces:
78         cv2.rectangle(im, (x, y), (x + w, y + h), (255, 0, 0), 2)
79         face = gray[y:y + h, x:x + w]
80         face_resize = cv2.resize(face, (width, height))
81         print(cv2.imwrite('% s/% s.png' % (path1, count), face_resize))
82         count += 1
83
84     cv2.imshow('OpenCV', im)
85     key = cv2.waitKey(10)
86 =     if key==27:
87         break
88
89     cv2.destroyAllWindows()
90

```

Face Recognition

```

93     if not os.path.exists('output'):
94         os.mkdir('output')
95
96     print("Loading model ...")
97     model = load_model(r"C:\Users\Varshitha V\Downloads\modelnew.h5")
98     Q = deque(maxlen=128)
99     vs = cv2.VideoCapture(video)
100    writer = None
101    (W, H) = (None, None)
102    count = 0
103    countv=0
104    while True:
105        # read the next frame from the file
106        (grabbed, frame) = vs.read()
107
108        # if the frame was not grabbed, then we have reached the end
109        # of the stream
110        if not grabbed:
111            break
112
113        # if the frame dimensions are empty, grab them
114        if W is None or H is None:
115            (H, W) = frame.shape[:2]
116
117        # clone the output frame, then convert it from BGR to RGB
118        # ordering, resize the frame to a fixed 128x128, and then
119        output = frame.copy()
120
121        frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
122        frame = cv2.resize(frame, (128, 128)).astype("float32")
123        frame = frame.reshape(128, 128, 3) / 255
124
125        # make predictions on the frame and then update the predictions
126        # queue
127        preds = model.predict(np.expand_dims(frame, axis=0))[0]
128        # print("preds",preds)
129        Q.append(preds)
130

```

Action Recognition Snippet 1

```

131     # perform prediction averaging over the current history of
132     # previous predictions
133     results = np.array(Q).mean(axis=0)
134     i = (preds > 0.50)[0]
135     label = i
136
137     text_color = (0, 255, 0) # default : green
138
139     if label: # Violence prob
140         text_color = (0, 0, 255) # red
141         countv+=1
142         print(countv)
143
144     else:
145         text_color = (0, 255, 0)
146
147     text = "Violence: {}".format(label)
148     FONT = cv2.FONT_HERSHEY_SIMPLEX
149
150     cv2.putText(output, text, (35, 50), FONT,1.25, text_color, 3)
151
152     # check if the video writer is None
153     if writer is None:
154         # initialize our video writer
155         fourcc = cv2.VideoWriter_fourcc(*"MJPG")
156         writer = cv2.VideoWriter("output/v_output.avi", fourcc, 30,(W, H), True)
157
158     # write the output frame to disk
159     writer.write(output)
160
161     # show the output image
162     cv2.imshow("frame",output)
163     key = cv2.waitKey(1)
164
165     # if the `q` key was pressed, break from the loop
166     if key == ord("q"):
167         break
168     # release the file pointersq

```

Action Recognition Snippet 2

```

171     print(countv)
172     if countv>0:
173         print("Violent")
174         print("\n")
175         url = 'sws'
176         account_sid = 'AC9b3f0311718ea6d49b54f1d83a983114'
177         auth_token = '988a50e8cd298a0e99e8dfbb84b1c079'
178         client = Client(account_sid, auth_token)
179
180
181     #location accessing
182     def getLocation():
183         gmaps = googlemaps.Client(key='AIzaSyCxxnYtwIqzWlAuPsdMw11Qktg07V5l5qI')
184
185         l = []
186         async def getCoords():
187             locator = wdg.Geolocator()
188             pos = await locator.get_geoposition_async()
189             l.append(pos.coordinate.latitude)
190             l.append(pos.coordinate.longitude)
191             return [pos.coordinate.latitude, pos.coordinate.longitude]
192
193         def getLoc():
194             return asyncio.run(getCoords())
195         print(getLoc())
196         getLocation.url = "https://google.com/search?q="+str(l[0])+','+str(l[1])
197         reverse_geocode_result = gmaps.reverse_geocode(tuple(l))
198         getLocation.final_address = reverse_geocode_result[0]['formatted_address']
199         print(getLocation.final_address)
200         print(getLocation.url)
201         #webbrowser.open_new(url)
202         print("GetLocation() starts...")
203         getLocation()
204         print("GetLocation() ends...")

```

Location Accessing

```

206     #sendMessage
207     def sendMessage():
208
209         text1 = "Hello,\nHere is the location where the incident was detected alongwith the map URL of the location.\nAddress: "
210         +getLocation.final_address+"\n\nLink:"+getLocation.url
211
212         message = client.messages.create(
213             body = text1,
214             from_='+15073997123',
215             to='+919731430189'
216         )
217         print(message.sid)
218
219         message1 = client.messages.create(
220             from_='whatsapp:+14155238886',
221             body=text1,
222             to='whatsapp:+919731430189'
223         )
224
225         print(message1.sid)
226
227
228     #Make calls to number
229     def makeCall():
230         call = client.calls.create(
231             url="https://handler.twilio.com/twiml/EHa30e7d15c246d02d472ab2784e6ea03a",
232             to="+919731430189",
233             from_="+15073997123"
234         )
235         print(call.sid)
236         print("\nMessage and call SIDs:")
237         sendMessage()
238

```

Code for Calls and Messages

```
242 =         for i in range(3):
243
244             playsound(r"C:\Users\Varshitha V\Downloads\mixkit-street-public-alarm-997.mp3")
245
246 =         if countv==0:
247             print("Non-Violent")
248
249             #writer.release()
250             vs.release()
251
252         V_path = r"C:\Users\Varshitha V\Videos\Main Dataset\Violence\V_19.mp4"
253
254         print_results(V_path)
```

Alarm Triggering